

A Survey on AI Agents for Data Management: Taxonomy, Progress, and Opportunities

ARNOB RAHEE, Bowling Green State University, USA

ARIJIT KHAN, Bowling Green State University, USA

1 Abstract

Autonomous AI agents powered by large language models (LLMs) have rapidly advanced in their ability to reason, plan, use tools, and iteratively refine their behavior. These capabilities position them to automate a broad spectrum of data management tasks long dominated by rule-based systems, manual scripts, and static optimizations. This survey provides a comprehensive review of recent LLM-based agentic systems across the data management life cycle, including data cleaning, ETL construction, database tuning, query optimization, fact-checking, and data validation. We analyze how these architectures employ multi-step planning, memory-augmented reasoning, tool invocation, and feedback-driven refinement to execute complex workflows from natural language instructions. Systems are grouped by the data management tasks they target and further examined through the lenses of orchestration patterns, autonomy levels, memory designs, and input modalities. For each category, we assess strengths, limitations, and emerging usage trends. The survey concludes by outlining open challenges and research directions—emphasizing the need for more reliable, explainable, safe, and scalable agentic systems—and highlights the potential of LLM-driven agents to deliver robust, trustworthy, and domain-specialized automation in data management. The online resources of this survey are available on [\[add github\]](#).

ACM Reference Format:

Arnob Rahee and Arijit Khan. 2026. A Survey on AI Agents for Data Management: Taxonomy, Progress, and Opportunities. *ACM Comput. Surv.* 1, 1, Article 1 (January 2026), 45 pages. <https://doi.org/XXXXXXX.XXXXXXX>

2 Introduction

The scale, heterogeneity, and rapid evolution of modern data ecosystems have made traditional data management workflows increasingly difficult. Core tasks such as data cleaning, integration, and querying remain time consuming and error prone, often requiring extensive manual effort and domain expertise. As data volume and diversity continue to grow, static rules, one-shot scripts, and brittle pipelines struggle to adapt to dynamic schema, complex logic, and dynamic workloads [4, 40, 125]. Recent position papers highlight a growing consensus that data management systems must become more autonomous, adaptive, and capable of interacting through natural language [5, 38, 76]. This has motivated the emergence of LLM-based AI agents that can plan multi-step workflows, invoke appropriate tools, and refine their actions through feedback, enabling them to handle heterogeneous data and complex decision logic that traditional systems cannot manage. While deep learning and LLMs have significantly advanced natural language understanding and

Authors' Contact Information: Arnob Rahee, arahee@bgsu.edu, Bowling Green State University, Bowling Green, Ohio, USA; Arijit Khan, arijitk@bgsu.edu, Bowling Green State University, Bowling Green, Ohio, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7341/2026/1-ART1

<https://doi.org/XXXXXXX.XXXXXXX>

pattern recognition [61, 63], they remain fundamentally stateless and reactive: they generate outputs but cannot autonomously plan, coordinate tools, or maintain long-horizon state across multi-step tasks. As a result, standalone LLMs struggle with the procedural, iterative, and tool-driven nature of real-world data management workflows, because the broader pipeline requires persistent state, error recovery, and adaptive decision making that reactive, stateless models cannot provide [85].

LLM-powered agents address these limitations through structured reasoning and iterative refinement mechanisms that improve accuracy and interpretability. Techniques such as chain-of-thought prompting [113] enable step-by-step reasoning, while ReAct-style loops [120] allow agents to alternate between planning, executing code, and learning from intermediate outputs. This iterative, feedback-driven process enables agents to decompose complex data management tasks into manageable subtasks and adjust their behavior based on observed outcomes. In contrast to earlier models that relied on one-shot prompting or rigid, hard-coded rules [8], these new agentic architectures are dynamic, resilient, and align well with the inherently noisy, evolving, and error-prone nature of real-world data management scenarios. Recent work on agentic data systems and autonomous data infrastructures underscores that this adaptivity makes LLM-based agents a strong foundation for scalable, trustworthy, and continuously improving data management workflows.

Despite rapid progress, research on agentic data management systems remains fragmented across isolated tasks and heterogeneous architectural choices, with no unified conceptual framework or systematic comparison. A comprehensive survey is therefore timely and well-aligned: to synthesize a fast-moving landscape, unify patterns, and clarify open challenges. Our survey provides such an integrative view, mapping the current landscape of LLM-powered data management agents and highlighting emerging research directions. We present the material in a way that is accessible to data, AI/ML, systems, and computing practitioners while maintaining scientific rigor through structured analysis. We examine each domain and architectural pattern in depth so readers can understand both the broader trajectory and the future potential of this rapidly evolving field.

Contributions. Our contributions are summarized as follows.

- (1) **Taxonomy.** The survey introduces a comprehensive taxonomy spanning eleven core data management problems—from foundational tasks such as data cleaning, entity resolution, and schema matching to emerging interfaces like text-to-SQL, pipeline automation workflows such as ETL, and semantic web and knowledge-centric applications.
- (2) **Agentic Paradigms.** This survey analyzes the major architectural paradigms used in agentic systems for data management. Rather than treating agents as monolithic black boxes, we decompose each system by its architectural organization, control flow, reasoning structure, memory design, and tool orchestration strategy. We focus on four dominant paradigms—single-agent tool-using architectures, manager-worker (hierarchical) designs, collaborative multi-agent systems, and planner-executor separations—and examine how each approach performs pre-execution planning, invokes tools, and, in multi-agent settings, coordinates interactions.
- (3) **Challenges and Opportunities.** Current agentic systems are still evaluated primarily with traditional benchmarks that measure only final outputs, leaving no unified framework for assessing the quality of multi-step reasoning or the correctness of tool invocation sequences. A similar gap exists in multi-agent settings: although systems adopt diverse coordination patterns such as manager-worker hierarchies or peer-to-peer collaboration, there are no established metrics for evaluating how effectively agents communicate or coordinate. These limitations also create clear opportunities: developing benchmarks that capture reasoning quality and collaboration effectiveness, designing more robust orchestration mechanisms, and building agents that generalize across tabular, graph, and semi-structured data. Addressing such challenges is critical for making agentic systems more reliable and scalable for real-world data management.

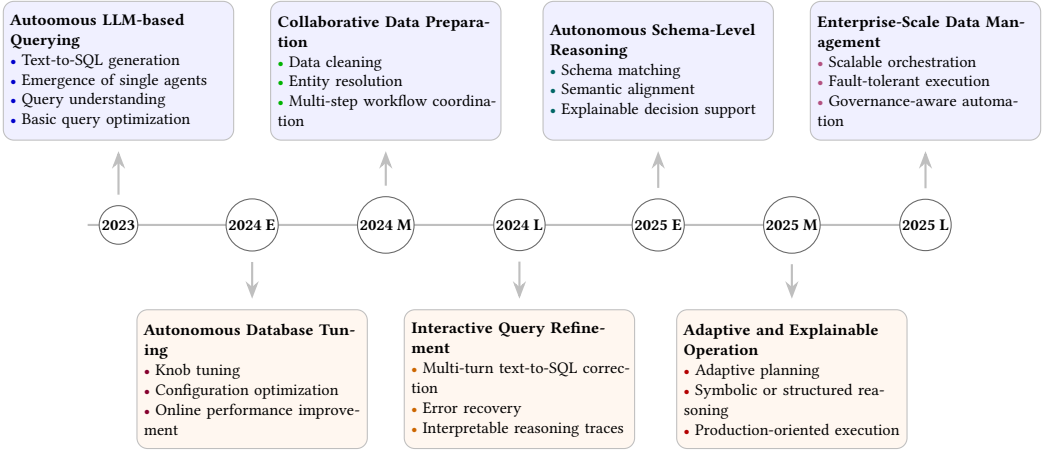


Fig. 1. Evolution of LLM-based agents for data management (2023–2025). ‘E’ denotes Jan–Apr, ‘M’ denotes May–Aug, and ‘L’ denotes Sep–Dec.

Roadmap. The remainder of the article is organized as follows. Section 3 outlines the survey scope, literature selection methodology, and the hierarchical taxonomy that structures the landscape into three clusters—Data Quality & Integration, Query & Optimization, and Governance & Knowledge—spanning eleven core data management tasks. Sections 4–14 each examine one task—data cleaning, entity resolution, schema matching, ETL automation, text-to-SQL, SQL optimization, database tuning, fact-checking, multimodal data management, data governance, and semantic web/knowledge graph engineering. For each, we compare single-agent and multi-agent architectures; analyze memory designs and orchestration patterns (e.g., iterative refinement loops, hierarchical manager-worker models, collaborative multi-agent protocols, planner-executor separations); evaluate performance across tabular, graph, and semi-structured data; and review available benchmarks, experimental setups, datasets, repositories, and external tools. Section 15 synthesizes cross-cutting challenges, research gaps, and emerging opportunities, highlighting needs in benchmarking, scalable multi-agent coordination, knowledge integration, and the development of more robust and reliable agentic architectures for real-world data management applications.

3 Survey Scope and Taxonomy

3.1 Literature Selection

This survey covers research published between 2023 and 2025, as demonstrated in Figure 1 (timeline), including both peer-reviewed papers and arXiv preprints. Works were included only if they (i) present a complete end-to-end agentic system, (ii) provide an evaluation framework that measures decision making quality—such as correct tool selection—and, for multi-agent settings, the effectiveness of inter-agent collaboration, and (iii) demonstrate a concrete data management application, such as data cleaning, integration, transformation, or query processing. Only papers satisfying all criteria were retained.

We conducted a comprehensive search across Google Scholar, the ACM Digital Library, IEEE Xplore, arXiv, and DBLP using keywords such as ‘LLM agent,’ ‘AI agent,’ ‘agentic system,’ ‘data management,’ ‘data cleaning,’ ‘text-to-SQL,’ ‘entity resolution,’ ‘schema matching,’ ‘ETL,’ ‘query optimization,’ ‘database tuning,’ ‘fact checking,’ ‘data governance,’ and ‘knowledge graph.’ This search yielded a broad set of candidate papers, from which we retained only those describing explicit

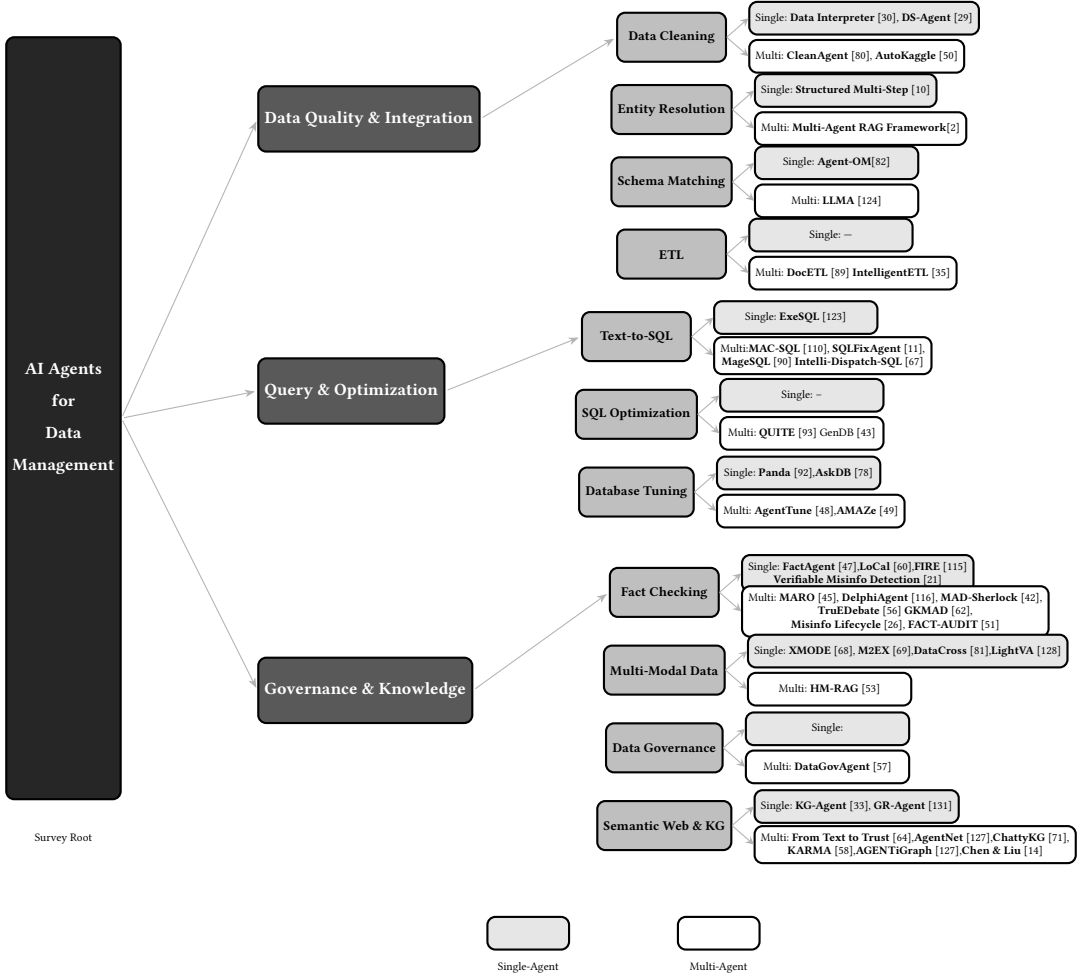


Fig. 2. Taxonomy of AI agents for data management tasks

agentic systems. Applying this filter resulted in a corpus of 60 papers featuring LLM-based agents for data management, all demonstrating end-to-end workflows with minimal human intervention.

3.2 Taxonomy

Figure 2 presents our hierarchical taxonomy, organizing the landscape into three functional clusters aligned with distinct operational objectives.

- **Data Quality & Integration:** Encompassing data cleaning, entity resolution, schema matching, and ETL automation—tasks focused on ensuring data accuracy, consistency, and interoperability.
- **Query & Optimization:** Covering natural language interfaces (Text-to-SQL), query optimization, and database configuration tuning—tasks focused on data accessibility and system performance.
- **Governance & Knowledge:** Addressing fact checking, multi-modal data management, data governance, and semantic web/knowledge engineering—tasks focused on metadata management, verification, and semantic reasoning.

Table 1. Comparison of survey coverage across recent LLM/agent surveys for data management.

Domain/Aspect	Tang et al.[96] (2025)	Zhou et al.[132] (2025)	Wang et al.[15] (2025)	Zhu et al.[133] (2025)	Our Survey (2026)
Data Quality & Integration					
Data Cleaning	✓	✓	Partial	✓	✓
Entity Resolution	×	×	×	Partial	✓
Schema Matching	×	×	×	Partial	✓
ETL Automation	×	✓	×	Partial	✓
Query & Optimization					
Text-to-SQL	✓	✓	×	✓	✓
SQL Optimization	×	✓	×	✓	✓
Database Tuning	×	✓	×	Partial	✓
Governance & Knowledge					
Fact Checking	×	×	×	Partial	✓
Multi-Modal Data	×	✓	×	Partial	✓
Data Governance	×	✓	×	Partial	✓
Semantic Web & KG	×	Partial	×	×	✓
Architectural Analysis					
Single-Agent Systems	✓	✓	✓	✓	✓
Multi-Agent Coordination	Partial	Partial	Partial	Partial	✓
Planning Mechanisms	✓	✓	✓	✓	✓
Tool Integration	✓	✓	✓	✓	✓
Memory Mechanisms	Partial	✓	×	Partial	✓
Evaluation & Benchmarks					
Domain-Specific Metrics	Partial	Partial	Partial	×	✓
Cross-Domain Comparison	×	×	×	×	✓

Across all eleven fields, we observe two dominant architectural paradigms: single-agent systems that rely on iterative reasoning-execution loops, and multi-agent systems that coordinate specialized roles through shared state and hierarchical planning. Sections 4-14 examine each field in depth, showing how these patterns manifest across different data management contexts.

3.3 Related Survey

To situate our work within the rapidly evolving landscape of LLM-based agents for data management, we review recent surveys in this area. Table 1 provides a comparative overview of coverage across three seminal works: Tang et al. [96] provide the first comprehensive survey specifically on LLM/Agent-as-Data-Analyst; Zhou et al. [132] present a systematic survey of LLM×Data interactions; Chen et al. [15] focus specifically on data science agents; while Zhu et al. [133] gives us a comprehensive picture of data agents with various autotomies ranging from L0 (no autonomy) to L5 (full autonomy). In contrast, our survey focuses exclusively on LLM-based agentic systems for eleven specific core data management tasks, providing dedicated per-task sections, fine-grained architectural comparisons, and cross-domain synthesis of challenges and opportunities.

While Tang et al. [96] focus primarily on data analysis tasks (visualization, statistical analysis, report generation) with emphasis on the data analysis workflow, they do not cover core data management operations such as entity resolution, schema matching, or database tuning. Zhou et al. [132] provide broad coverage of LLM applications in data management but treat each domain in isolation without systematic analysis of architectural patterns (single-agent vs. multi-agent)

that generalize across tasks. Chen et al. [15] narrow their scope to data science competitions and ML workflows, omitting traditional data management concerns like ETL, data governance, and semantic web.

Our survey extends these works by providing: (1) *comprehensive coverage* of all eleven data management domains shown in Figure 2; (2) *unified architectural taxonomy* analyzing how single-agent and multi-agent patterns manifest across diverse tasks; and (3) *cross-domain synthesis* identifying common challenges (evaluation fragmentation, tool integration, hallucination risks) and opportunities (hybrid architectures, standardized benchmarks) that transcend individual domains.

4 LLM Agents for Data Cleaning

Data cleaning is a foundational step in the data science lifecycle. AI agents can accelerate cleaning by automating both syntactic fixes (e.g., format normalization and missing value imputation) and semantic corrections (e.g., unit mismatches, cross-field inconsistencies, and contextual outlier detection). Because practitioners commonly report spending a majority of project time on data preparation [25], recent research has shifted toward agentic systems that move beyond rule-based scripts. These systems leverage large language models to perform multi-step planning, tool-based execution, and feedback-driven refinement, enabling them to handle heterogeneous formats, infer contextual anomalies, and synthesize multi-step cleaning workflows from high-level natural language instructions. In particular, standalone, goal-directed agentic architectures—often inspired by AutoGPT-style autonomous execution loops—are being explored to automate and scale complex data cleaning tasks without relying on external orchestration or retrieval frameworks.

4.1 Survey

The emergence of large language models has enabled the rise of agentic AI systems that move beyond stateless, single-step prompting toward multi-step planning, tool use, and feedback-driven refinement—capabilities that make them well-suited for iterative tasks such as data cleaning. Reasoning-capable LLMs have expanded the design space for autonomous data science agents (Table 2), leading to diverse choices in agent structure, planning strategies, memory mechanisms, and tool integration. *Single-agent systems* such as **Data Interpreter** [30], **DS-Agent** [29], **Data-Copilot** [126], and **Lambda** [95] operate through iterative reasoning–execution loops, differing primarily in how they store and reuse information and how they sequence workflow steps. In contrast, **CleanAgent** [80], **AutoKaggle** [50], and **AutoML-Agent** [102] employ *multi-agent architectures* with manager-driven or hierarchical coordination across subtasks such as schema understanding, preprocessing, modeling, and evaluation. Their *tool scopes* also vary: **CleanAgent** focuses on declarative standardization APIs, whereas **AutoKaggle** and **AutoML-Agent** incorporate broader data science and AutoML toolchains for end-to-end pipelines. Finally, these systems operate across different *data regimes*—from messy user tables in **CleanAgent** to competition-scale datasets in **AutoKaggle** and large domain-specific corpora in **Data-Copilot**—leading to distinct trade-offs in autonomy, latency, and reliability.

Table 2. AI Agents for Data Cleaning

Method	Agent Type	Orchestration	Planning	Memory	External Tools	Data Cleaning Strategy	Data Types
Data Interpreter [30]	Single-agent	Single-agent loop	Stepwise reasoning-execution	Execution traces	Python execution environment, code execution runtime	Preprocessing, error detection	Tabular
DS-Agent [29]	Single-agent	Closed-loop execution	Case-guided planning	Case memory	Code generation/execution environment, Kaggle case library	Preprocessing, feature cleaning	Tabular
Data-Copilot [126]	Single-agent	Sequential execution	Workflow-level planning	Workflow context	Pre-designed compiler-validated interface tools, visualization/table interfaces	Data preparation, transformation	Tabular
Lambda [95]	Single-agent	Autonomous loop	Implicit step planning	Execution context	External models and algorithms via Knowledge Integration Mechanism	Data preprocessing	Tabular
CleanAgent [80]	Multi-agent	Manager-worker coordination	Explicit multi-step planning	Conversation history	Dataprep.Clean, Python standardization APIs	Standardization, normalization, error correction	Large Tabular
AutoKaggle [50]	Multi-agent	Hierarchical coordination	Pipeline-level planning	Shared state	Universal data science toolkit, code execution/debugging, unit testing	Preprocessing within ML pipelines	Tabular
AutoML-Agent [102]	Multi-agent	Pipeline execution	End-to-end planning	Pipeline artifacts	Retrieval-augmented planning module, execution/verification pipeline	Preprocessing for AutoML	Tabular

Data Interpreter [30] and **DS-Agent** [29] are primarily single-agent systems equipped with Python execution environments, enabling them to analyze datasets, generate and run code, apply transformations, and iteratively refine their actions based on execution feedback. Following a *ReAct-style paradigm*, these agents alternate between reasoning and acting: They first form a high-level understanding of the task, then write and execute code, evaluate the results, and determine subsequent steps. This iterative think-act-revise loop makes them particularly effective for data cleaning and other tasks that require progressive correction rather than a single, one-shot solution.

In contrast, **CleanAgent** [80] is purpose-built for data cleaning and standardization, employing a multi-agent architecture in which specialized agents collaborate to infer column semantics, generate cleaning code, and iteratively validate results through execution feedback. Following a *manager-worker interaction pattern*, the *manager agent* oversees the overall cleaning objective and delegates subtasks to dedicated components. A *column-type annotator* first infers semantic column types, which are then used by the *programmer agent* to produce appropriate cleaning code. The *executor agent* runs this code and reports results or errors back to the manager, enabling further refinement when needed. **CleanAgent** thus demonstrates a concrete instantiation of coordinated agentic workflows for real-world data cleaning tasks.

End-to-end agentic workflows have also been explored through multi-agent architectures in which different agents specialize in subtasks such as preprocessing, modeling, and evaluation. Systems like **AutoKaggle** [50] and **AutoML-Agent** [102] exemplify this approach by coordinating specialized agents across *stages of the data science pipeline*, particularly for data preprocessing and

data quality improvement. Similarly, **Data-Copilot** [126] and **Lambda** [95] introduce autonomous agents that translate high-level user intent into multi-step data preparation workflows, decomposing tasks into subtasks, selecting appropriate tools, and adjusting actions based on intermediate results. These systems follow *goal-oriented, multi-step execution patterns* reminiscent of *AutoGPT-style* [91] agents, which independently plan, act, and refine their strategies with minimal human intervention.

Finally, several papers [16, 30] analyze the capabilities and limitations of LLM-based agents in practical data analysis scenarios. Evaluations show that while LLM agents work well in code generation and exploratory analysis, they struggle with reliability and consistency without structured execution control and monitoring [17]. Systems like **InsightPilot** [23], **WaitGPT** [114], and **Jupytera** [111] illustrate the desirability of tighter integration among agent reasoning, data operations, and monitoring to support robust and transparent agentic data workflows.

4.2 Metrics and Benchmark for performance

Evaluating agentic AI systems for data cleaning requires attention to both data quality outcomes and the agent’s execution process. As these systems operate through multi-step, non-rule-based workflows, their assessment is multi-dimensional. For systems dedicated to cleaning, output correctness remains central. **CleanAgent**, for example, reports *Cell-Level Matching Rate* (CLMR) to quantify the proportion of correctly standardized or repaired cells relative to ground truth, and measures latency to report *coordination overhead*, *code generation time*, and *execution cost*.

General-purpose agentic data science platforms are typically evaluated at the *task level*. **Data Interpreter** measures completion and correctness on predefined analysis tasks, while **DS-Agent**, **AutoKaggle**, and **AutoML-Agent** rely on success rates, normalized task scores, or ranking metrics across benchmark tasks and competitions—where preprocessing quality often plays a decisive role. Increasingly, agent-centric metrics are also used to assess planning reliability and execution robustness, including measures of successful execution, failure recovery, and consistency across repeated runs [17, 95, 126]. Runtime errors, incorrect outputs, and assertion failures serve as valuable *feedback signals* for improving agent behavior.

Recent work further explores *LLM-specific* and *workflow-aware evaluation*, using precision, recall, and F1-scores to compare generated cleaning workflows against ground truth operations. Although no standardized benchmark has yet emerged, these efforts reflect a growing trend toward principled, operation-level assessment of agentic data cleaning systems. Current evaluation practices remain fragmented across platforms, but the diversity of metrics underscores the need for shared criteria that jointly capture data quality, reasoning traceability, and execution efficiency.

4.3 Data and Tools

Agentic AI systems for data cleaning and preparation are predominantly evaluated on *real-world tabular datasets* that exhibit practical challenges such as missing values, inconsistent formats, duplicates, and semantic errors. These datasets typically appear as CSV files, spreadsheets, or relational tables, and are either user-provided or sourced from benchmark repositories such as OpenML [105] and classic data cleaning benchmarks including Katara [18] and HoloClean [83]. Several systems further rely on error-injected relational tables to assess data repair, constraint-violation detection, and semantic consistency within full data science pipelines.

A first class of systems operates directly on raw user-provided tabular data. **CleanAgent**, **Data Interpreter**, **DS-Agent**, and **Lambda** exemplify this regime, targeting messy address files, customer records, administrative tables, and other heterogeneous datasets rife with missing values and semantic inconsistencies. These agents perform normalization, standardization, error detection, and feature cleaning through flexible action sequencing and tool selection driven by the characteristics of the input data.

A second class of systems integrates data cleaning into broader ML workflows. Multi-agent systems such as **AutoKaggle** and **AutoML-Agent** operate over competition-style or pipeline-oriented datasets, including *House Prices* [37], *Titanic* [36], and *IEEE-CIS Fraud Detection* [32]. These systems couple preprocessing, feature engineering, and validation with model training, emphasizing end-to-end automation where cleaning decisions directly influence downstream performance. **DataCopilot** and **DeepAnalyze** extend this paradigm to enterprise-scale analytical tables, including business records and domain-specific datasets with temporal or structural dependencies.

A third group of systems addresses heterogeneous data sources and distributed cleaning tasks. Systems such as **Autonomous Data Agents** and **R&D-Agent** decompose workflows across specialized agents that coordinate through shared state, memory, and intermediate artifacts, enabling robustness when individual components encounter failures.

Across these settings, agentic systems rely primarily on *Python execution environments* for iterative reasoning and action. Agents synthesize and execute code for inspection, transformation, and validation, using execution feedback to refine subsequent steps [29, 30, 95]. Most systems employ general-purpose data science libraries such as **Pandas** [65] and **NumPy** [104], which offer flexibility but can be brittle under unexpected data conditions. In contrast, **CleanAgent** introduces higher-level *declarative data cleaning APIs* to standardize dates, categorical values, and other fields, reducing reliance on low-level scripting. Pipeline-oriented systems such as **AutoKaggle** and **AutoML-Agent** incorporate validated preprocessing and feature engineering toolchains to support robust end-to-end workflows.

Recent benchmark efforts further expand the landscape of agentic data cleaning evaluation. **AutoDCWorkflow** [46] provides multi-step cleaning workflows with operation-level annotations and diverse error types, enabling assessment of both cleaning accuracy and workflow correctness. The ALASKA benchmark for data integration [20] and critical analyses by Papadakis et al. [73] explore intentional error injection to systematically evaluate robustness. Collectively, these datasets and tools highlight emerging trends toward standardized, agent-aware evaluation frameworks for LLM-based data cleaning.

4.4 Future Direction

Agentic data cleaning systems, despite recent advances, still face significant challenges that limit their reliability and comparability. Benchmarks and metrics remain fragmented, making cross-system evaluation difficult, while the multi-step, tool-driven nature of agentic workflows introduces latency and reliability issues when execution errors cascade across steps. Addressing these gaps will require consolidated and standardized evaluation frameworks, stronger robustness guarantees, and deeper integration between data-centric principles and agent-based reasoning. Equally important are advances in explainability and human-in-the-loop collaboration, enabling agents to justify their cleaning decisions and incorporate user guidance when necessary. Additional open problems include ensuring safe execution, scaling to larger and more diverse datasets, enabling cross-domain generalization, supporting long-term learning, and establishing standardized representations for reasoning traces and collaborative workflows.

5 LLM Agents for Entity Resolution

Entity Resolution (ER)—the task of identifying records that refer to the same real-world entity across heterogeneous data sources—has traditionally relied on handcrafted similarity metrics, blocking heuristics, and supervised matching models, all of which struggle with schema heterogeneity, semantic ambiguity, and the evolving nature of real-world data. Emerging LLM-based agentic systems address these limitations by decomposing ER into specialized sub-tasks, orchestrating tool use for external validation, and dynamically adapting strategies under uncertainty and cost constraints.

Table 3. LLM Agents for entity resolution

Method	Agent Type	Orchestration	Planning	Memory	External Tools	Matching Strategy	Data Types
Multi-Agent RAG Framework [2]	Multi-agent	LangGraph (4 agents)	Sequential + conditional	Shared AgentState (graph)	Vector DBs, LLM APIs, phonetic encoders	Hybrid deterministic + RAG reasoning	Structured, temporal
Structured Multi-Step [10]	Single agent	Prompt chaining	3-step decomposition	Conversation context	None explicit	Token-attribute-entity reasoning	Structured, textual
Harmonia [86]	Single/Multi-agent	Interactive synthesis	Pipeline composition	Task state stack	Harmonization primitives, UI	Human-agent collaborative	Structured, clinical

These multi-step planners integrate token-level similarity signals, attribute-level importance, and relational dependencies while maintaining shared state across reasoning steps. By combining retrieval-augmented generation, coordinated multi-agent workflows, and human-in-the-loop verification, LLM agents deliver more accurate and interpretable ER pipelines suited for high-stakes domains such as national censuses, medical analytics, and government data integration.

5.1 Survey

LLM-based agents facilitate *autonomous ER workflows* where an agent orchestrates the use of various tools to perform entity matching, verification, and clustering tasks. We categorize such systems based on the agentic architectural patterns: multi-agent orchestration systems, structured reasoning agents, interactive pipeline synthesis agents, and cost-aware verification agents. Table 3 summarizes and compares several prominent LLM-based agent systems for entity resolution across key architectural dimensions, including agent type, orchestration mechanism, planning strategy, memory usage, external tools integration, matching approach, and supported data types, highlighting the diversity from multi-agent coordination (e.g., the LangGraph-based framework in [2]) to single-agent structured reasoning and cost-optimized verification methods.

Multi-agent orchestration systems. These represent the most advanced agentic architectures proposed for ER. Althaf et al. [2] introduce a multi-agent RAG framework that extends single-LLM solutions by orchestrating specialized agents using LangGraph. The system decomposes entity resolution into four coordinated agents: (a) *Direct Matcher*, which performs deterministic name-based matching; (b) *Indirect Matcher*, which exploits transitive and relational signals; (c) *Household Matcher*, which clusters records using address and residence information; and (d) *Move Detector*, which identifies temporal moves across datasets. Each agent employs customized Retrieval-Augmented Generation (RAG) pipelines and hybrid data cleaning methods that combine deterministic rules with LLM-based contextual disambiguation. The LangGraph orchestration layer maintains shared state through an AgentState object, enabling structured inter-agent communication, selective re-computation, and full traceability of decisions. This modular design achieves 94.3% accuracy while reducing LLM API calls by 61% relative to single-LLM baselines, demonstrating clear gains in efficiency, explainability, and maintainability.

Structured reasoning agents. The central idea behind these agentic approaches is to treat entity matching as a sequence of structured cognitive steps rather than a single classification task. Bopardikar et al. [10] propose a three-stage agentic framework in which the agent (i) identifies matching and non-matching tokens across the two records, (ii) determines the attributes most influential to the decision, and (iii) predicts whether the records refer to the same entity. The framework employs Chain-of-Thought (CoT) reasoning and uses an LLM-based agent to generate explanations for each decision. The authors also explore a debate-based strategy in which two contrasting agents produce pro and con arguments that are subsequently reconciled to yield a more robust match decision. Compared to single-prompt LLM matchers, these structured-reasoning agents expose intermediate reasoning steps, improving transparency, validation, and auditability.

Interactive pipeline synthesis agents. These agentic approaches automate complex data harmonization workflows by integrating LLM-driven reasoning with human expertise. Santos et al. [86] introduce *Harmonia*, an agentic system that combines an LLM reasoning engine, an interactive interface, and a library of harmonization primitives to synthesize task-specific workflows based on user intent and data characteristics. Rather than executing a fixed pipeline, *Harmonia* composes primitives such as schema matching, entity resolution, value mapping, and supports iterative refinement that allows domain experts to guide key decisions while automating underlying transformations. Demonstrations on clinical and administrative datasets illustrate the value of LLM-based agentic systems as collaborative partners for scalable, adaptive harmonization.

5.2 Metrics and Benchmark

Beyond matching accuracy, the performance of LLM-based ER agents should be evaluated along dimensions such as reasoning quality, cost efficiency, interpretability, uncertainty reduction, and multi-agent coordination. While *matching accuracy* remains central—typically measured using precision, recall, and F1-score on held-out data—recent systems report strong results, including 94.3% accuracy and 93.4 F1-score on synthetic administrative data [2] and clear gains over zero-shot baselines with structured reasoning [10]. *Cost efficiency* can be assessed through token usage, API-call frequency, and average cost per resolved entity; multi-LLM agents achieve up to 52% faster runtime and 62% lower token usage by sharing state and selectively retrieving evidence [2]. *Interpretability and traceability* involve evaluating the completeness of reasoning traces, confidence calibration, and human-judged explanation quality, with structured JSON outputs yielding high interpretability scores [2]. *Uncertainty reduction* metrics, such as entropy decrease per query, capture the efficiency of cost-aware verification, where agents converge to correct partitions with minimal LLM queries. *Agent coordination efficiency* measures correctness of state transitions, communication overhead, and redundancy reduction in multi-agent settings. Traditional ER metrics, e.g., reduction ratio and pairs completeness remain useful but should be adapted to reflect agent-specific behaviors (such as precision of a Direct Matcher versus recall of an Indirect Matcher). Current benchmarks rely on synthetic administrative datasets (e.g., S12PX) [2], standard ER corpora, WDC Products [77], and domain-specific clinical harmonization data [86], but standardized datasets that capture multi-hop reasoning, tool-use accuracy, cost-constrained performance, and human-agent interaction remain an open challenge.

5.3 Data and Tools

Existing agent-based ER systems tackle resolution tasks across diverse data modalities including relational tables, semi-structured documents, and temporally ordered event streams. Unlike classical ER methods that assume clean, aligned schemas, modern deployments operate over schema-flexible data lakes where attribute semantics are often ambiguous, inconsistent, or poorly defined across sources. Althaf et al. [2] address these challenges using a multi-agent RAG framework for synthetic census and administrative data in which attributes are corrupted by typos, nulls, and formatting inconsistencies. *Harmonia* [86] targets clinical datasets characterized by heterogeneous medical terminologies and protocol-specific variation.

LLM-based ER pipelines rely on a broad tool ecosystem. Vector databases (e.g., FAISS, Qdrant) store entity embeddings to support semantic search and RAG grounding. LLM APIs including GPT-4, Mistral, Gemini, and local models such as Llama 3.2 and DeepSeek perform generative reasoning, with model selection driven automatically by task complexity and data sensitivity constraints [2]. Deterministic preprocessing tools (e.g., Soundex, NYSIIS, address normalization libraries, Jaro-Winkler) complement LLM reasoning to improve interpretability and efficiency. Orchestration frameworks such as LangGraph and LangChain coordinate multi-agent workflows, maintain shared

state, and execute graph-structured plans in which agents are nodes and data dependencies form edges. Interactive interfaces (e.g., Streamlit) enable human oversight and mixed-initiative correction [2, 86]. Finally, code execution environments allow agents to run data processing scripts and similarity functions they generate.

LLM ER systems use memory components to maintain reasoning context. Single-agent systems rely on conversational context objects, while multi-agent systems maintain structured global state. In the multi-agent RAG framework of [2], each agent's state is stored in a shared `AgentState` object managed by LangGraph, enabling atomic state access and fine-grained provenance tracking across agent interactions.

5.4 Future Directions

LLM-based agents for entity resolution still face several unresolved challenges. Scalability remains unproven, as current multi-agent systems have only been tested on datasets of a few hundred records, leaving their behavior on million-record workloads unclear [2]. Standardized benchmarks are also lacking; existing static pairwise matching datasets do not capture the multi-step planning, tool use, cost constraints, and human interaction that characterize agentic ER pipelines. Advances in adaptive reasoning are needed as well, since today's agents rely on fixed prompts rather than learning strategies through RLHF or data-dependent task decomposition [2]. Human-agent collaboration remains limited to verification rather than mixed-initiative dialogue, where agents proactively seek clarification or propose alternative hypotheses [86] requiring agents that integrate text, structured data, and images. Finally, ensuring trustworthiness is essential: ER agents must provide calibrated uncertainty estimates, detect demographic biases in match decisions, and generate explanations accessible to non-technical stakeholders in domains such as census analysis and healthcare.

6 LLM Agents for Schema Matching

Schema matching—identifying correspondences across heterogeneous schemas—was traditionally performed manually, through rule-based systems, or via supervised models trained on past alignments, all of which fall short in data lake settings marked by semantic variability and evolving vocabularies. Recent LLMs improve semantic matching through vector similarity and few-shot prompting, but LLM-based agents extend this capability by adding planning, tool orchestration, and collaborative reasoning. In these agentic frameworks, matching becomes an active, multi-step search process: agents plan retrieval and comparison tasks, decompose them into executable sub-tasks, negotiate to reconcile partial alignments, and use external tools for validation. This shifts schema matching from a static similarity computation to a dynamic workflow involving information retrieval, hypothesis formation and testing, and consensus building, yielding more robust results in complex environments.

6.1 Survey

Our inspection indicates that LLM-based agents for schema matching remain at an early stage, and we identify two emerging architectural patterns: the *Siamese tool-augmented agent* and the *negotiation-based multi-agent system*. As summarized in Table 4, these patterns reflect the growing diversity of agentic approaches to schema and ontology matching, illustrating how representative systems differ across key architectural dimensions such as tool integration, coordination mechanisms, and interaction models.

Siamese tool-augmented agents. Agent-OM [82] introduces a full LLM-agent framework for ontology and schema matching, built around a Siamese architecture with two coordinated peers—the Retrieval Agent and the Matching Agent—that share a hybrid memory structure but follow distinct planning processes. The Retrieval Agent decomposes the alignment goal into subgoals (e.g.,

Table 4. LLM Agent-based Schema Matching [last table column is inconsistent w.r.t. previous tables]

Method	Agent Type	Orchestration	Planning	Memory	External Tools	Matching Strategy	Data Types
Agent-OM [82]	Multi-agent (retrieval + matching agents)	Functional delegation between agents	CoT-based ontology matching decomposition	Hybrid storage (dialogue history + relational/vector DB)	Ontology retrieval modules, entity matching tools, RAG/ICL retrieval tools	Evidence retrieval followed by candidate alignment evaluation	Ontology / Knowledge Graph
LLMA [124]	Multi-agent (peer agents)	Peer negotiation workflow	Dialogue-based iterative reasoning	Shared dialogue history	Schema registries, agent communication interfaces	Argumentative negotiation and critique until consensus	Relational schema / Ontology

identifying candidate matches, retrieving metadata, gathering semantic evidence) and executes them using specialized retrieval tools, storing all retrieved evidence in a shared hybrid database store that serves as working memory. The Matching Agent then consumes this evidence, using its own planner to orchestrate Matching Validators and Candidate Summarizers to assess match confidence. This separation of retrieval and judgment enables concurrent evidence gathering while maintaining coherent semantic interpretation across schemas. Overall, the architecture demonstrates how schema matching benefits from a division of labor across specialized agents, mirroring human workflows in which data administrators analyze sources before final alignment decisions are made.

Argumentation-enabled negotiation in multi-agent systems. A different, negotiation-based approach has recently been proposed by LLMA. [name?] [124]. In this framework, multiple autonomous peer agents perform tasks similar to the function-specific agents in Agent-OM, but without the oversight of a central controller. Each agent independently proposes correspondences, evaluates the proposals of others, and engages in structured dialogical interactions to reach agreement on the final alignment. The architecture equips each agent with its own belief state over possible correspondences and enforces a shared dialogue protocol inspired by Dialogue Game Theory. While LLMs support the agents' *cognitive functions*, such as interpreting schema elements, generating candidate mappings, and computing semantic similarity, the agents themselves make *decisions* about proposing, accepting, or rejecting correspondences based on dialogue history. The protocol also handles cases where consensus cannot be reached, involving the user by marking uncertain mappings. This stands in contrast to the *functional specialization* paradigm of Agent-OM, where different agents are assigned distinct tools and responsibilities to reduce redundant computation. In the *redundant negotiation* paradigm of LLMA. [name?], agents share the same tools and capabilities, increasing robustness by identifying ambiguous cases where agreement fails and additional evidence or user input is required. Both systems address key limitations of monolithic LLM matchers, which suffer from fixed context windows, hallucinated mappings, and static reasoning without validation. Agent-OM mitigates these issues through tool-augmented retrieval and validation, while the negotiation-based system reduces hallucinations through adversarial cross-checking of proposed mappings, thereby lowering false positives.

6.2 Metrics and Benchmark

Classical alignment measures, *precision*, *recall*, and *F-measure* remain central, and both Agent-OM and negotiation-based systems report competitive performance on standard **OAEI** benchmarks. Agent-OM leverages extensive evidence retrieval to improve F-measure, while negotiation-based approaches are additionally assessed through *consensus rates*, reflecting the proportion of alignments resolved autonomously by agents. Beyond accuracy, recent work introduces efficiency metrics such as the number of tool invocations (indicating planning quality), the number of dialogue rounds in negotiation (indicating argumentation efficiency), and the *autonomy rate*, which measures the fraction of correspondences resolved without human intervention. Robustness is evaluated through measures such as *drift adaptation rate*, capturing how quickly agents adjust mappings after schema

changes, and *hallucination rate*, quantifying unsupported or spurious correspondences—typically lower in multi-agent settings due to cross-agent validation. Despite these advances, state-of-the-art benchmarks remain insufficient: while OAEI tracks (Anatomy, Conference, LargeBio) and other datasets (e.g., SMB, geospatial and biomedical ontologies) support alignment evaluation, none assess planning quality, tool usage, or coordination effectiveness. We, therefore, highlight the need for new benchmarks that incorporate dynamic schema drift scenarios, tool availability constraints, and multi-agent coordination challenges.

6.3 Data and Tools

Existing agent-based schema matching operates directly on heterogeneous schema artifacts including relational database DDLs, OWL ontologies, JSON Schemas, and XML Schema Definitions (XSDs)—and, unlike traditional matchers, consumes not pre-processed feature vectors, but a suite of APIs for accessing schema repositories. Central to this infrastructure are the *External Tools* used in Agent-OM: the *Metadata Retriever*, which extracts structural information (tables, columns, constraints) from database catalogs; *Lexical Retrievers*, which compute edit distance and token similarity for initial candidate filtering; *Semantic Retrievers*, which use LLMs to assess semantic similarity when lexical cues are insufficient; and the *Hybrid Database Store*, a combined relational and vector database serving as shared working memory for retrieval and matching. *Validation Tools* verify proposed mappings by checking type compatibility, instance counts, and related constraints. In negotiation-based approaches, communication protocols play an even more prominent role, orchestrating interactions among agents and enabling them to query schema registries to substantiate claims made during argumentation. LLM infrastructure underpins the reasoning capabilities of these systems: in Agent-OM, LLMs guide planning decisions (e.g., selecting the next retrieval tool) and perform validation, while in negotiation frameworks they generate natural language arguments for or against candidate correspondences. Crucially, these are not isolated LLM calls but chained interactions where model outputs determine subsequent tool use or dialogue actions.

6.4 Future Directions

The literature on multiagent-based schema matching remains in an early stage, and several important research challenges have begun to surface. One open direction concerns *multi-schema alignment*, where more than two schemas must be reconciled simultaneously. Existing agent architectures primarily support pairwise matching, leaving open the question of how to coordinate alignments across larger sets of schemas. This gap highlights the need for mechanisms such as *coalition formation*, in which agents representing different data sources collaborate to establish a shared, globally consistent alignment.

A second challenge involves *lifelong schema matching*. Current systems typically assume that data sources and matching tasks arrive in discrete batches. In real-world data lakes, however, schemas and tasks evolve continuously. Future agents will need to track schema versions, detect drift, trigger re-matching when necessary, retain useful prior knowledge, and apply incremental matching strategies to reduce computational cost.

Another promising direction is enabling more flexible *tool learning and tool growth*. Baseline agents rely on a fixed set of predefined tools for retrieval and validation. Next-generation agents should be capable of discovering, adapting to, or even inventing new tools—such as unfamiliar APIs, unseen database schemas, or domain-specific knowledge graphs—without manual configuration.

A further issue concerns *explainability and trust*. Although agent systems generate rich intermediate artifacts, such as retrieval traces and dialogue histories, these outputs are not yet optimized

Table 5. LLM-agent systems for ETL tasks

Method	Agent Type	Orchestration	Planning	Memory	Scalability	External Tools	Data Type
DocETL [89]	Multi-agent	Agent-based rewrite generation + validation loop	Agent-driven pipeline rewriting with plan evaluation and optimization search	Not explicit in paper	High (document-scale optimization and execution)	YAML DSL, Jinja templating, validation prompts, Python implementation, LLM tool-calling models	Unstructured / semi-structured documents
IntelligentETL [35]	Multi-agent	Primary-agent routing across SQL, RAG, and web-search tools via LangGraph state graph	Task routing across specialized agents (SQL, retrieval, web search)	LangGraph MemorySaver checkpointing + knowledge/state graph retention	Enterprise-scale document ingestion + relational processing	LangChain, LangGraph, Chroma, Tavily, GPT models, pdfplumber, camelot, pandas, SQLite	PDFs, CSV/XLSX/TXT, structured + unstructured enterprise data

for human consumption. Research on *argumentation summarisation* is needed to distill multi-agent reasoning into clear, human-interpretable justifications suitable for data governance and stewardship.

Finally, we underscore the need to formalize *hybrid human-agent workflows*. Optimal matching will likely combine automated agents for routine, high-confidence correspondences with human oversight for ambiguous or semantically complex cases. Determining principled policies for when to escalate decisions from agents to humans, based on uncertainty estimates, cost considerations, and governance requirements, remains an open and important problem.

7 ETL Problems using LLM Agents

ETL (Extract, Transform, Load) is a foundational component of data warehousing, business intelligence, and modern data pipelines. Traditional ETL development is highly manual and error-prone in the face of schema evolution or unstructured inputs; also scales poorly across heterogeneous or real-time data sources. Recent work shows that LLM-powered agents can mitigate these limitations by enabling autonomous reasoning, tool use, planning, task decomposition, and self-optimization of complex pipelines, particularly when handling semi-structured or unstructured data.

Emerging research introduces LLM-agent systems that move beyond simple prompt-based invocation. These systems orchestrate multi-step workflows, rewrite or refine pipelines for correctness, integrate specialized sub-agents (e.g., SQL generation, retrieval, or validation agents), and even serve as benchmarks for evaluating end-to-end autonomy in ETL.

7.1 Survey

Table 5 provides a comparative overview of the core LLM-agent systems for ETL tasks.

DocETL [89] is an explicitly agentic, declarative framework for LLM-based document ETL. Users specify pipelines in YAML using natural language semantic operators, while a dedicated optimizer applies agentic query rewriting (thirteen novel rewrite directives) and a “gleaning” procedure that separates generation and validation agents to iteratively refine plans and mitigate hallucinations or incomplete extraction. Across clause extraction, theme analysis, and related tasks in long legal and medical documents, DocETL delivers 25-80% improvements in F1 and recall over strong baselines. The system is open-source (over 1.7k GitHub stars) and includes an interactive playground UI.

IntelligentETL [35] is a full multi-agent LLM system for enterprise-scale unstructured data ETL. A central supervisor (GPT-4o-mini) orchestrates a LangGraph-based knowledge graph state machine that routes tasks to specialized agents: an SQL agent for relational joins and aggregations, a RAG agent for sequential text and table extraction from PDFs using Chroma embeddings, and a Tavily-powered web search agent for external context. The system ingests PDFs, extracts both tabular and free-text content, loads them into a unified SQLite relational schema, and supports

direct SQL querying. It achieves 100% accuracy on relational tasks and surpasses prior tools such as Unstract [134][cite] in mixed-data enterprise settings.

Together, these systems illustrate the shift from single-prompt LLM augmentation to genuinely agentic ETL orchestration: DocETL through logical rewriting and validation loops, and IntelligentETL through persistent-state, supervisor-driven multi-agent routing.

7.2 Metrics and Benchmark

Evaluating LLM-agent ETL systems requires combining classical pipeline metrics with agent-centric measures that capture autonomy, adaptability, and reasoning quality. Traditional indicators—throughput (records per second), end-to-end latency, and resource utilization—remain foundational, while accuracy and completeness are especially critical for unstructured and mixed-format inputs. DocETL reports 25-80% gains in F1 and recall for complex clause extraction and theme analysis via its agent-guided rewriting and validation loops [89], and IntelligentETL achieves 100% correctness on relational operations after ingesting PDFs containing both tables and free text [35]. Additional dimensions assess development efficiency (time-to-pipeline, generated code correctness) and agentic capabilities such as rewrite quality, schema inference precision, dynamic adaptation, and error recovery latency.

Traditional benchmarks like TPC-DI evaluate only static performance and ignore agentic behaviors. ELT-Bench [34] fills this void as the first dedicated end-to-end benchmark for AI agents building complete ELT pipelines, testing schema discovery, code generation, tool interaction, and transformation correctness across 100 realistic pipelines spanning 835 source tables. Current frontier agents still achieve only low single-digit full-model success rates at high cost and step counts, underscoring the remaining challenges. DocETL and IntelligentETL provide complementary task-specific accuracy suites for document-centric and enterprise mixed-data scenarios, yet broader standardized benchmarks covering real-time streams, self-healing, and multi-agent coordination are still needed.

7.3 Data and Tools

DocETL operates exclusively on long-form unstructured and semi-structured documents that frequently exceed standard LLM context windows. Representative datasets include the Contract Understanding Atticus Dataset (CUAD) with 510 expert-annotated legal contracts, concatenated Steam video game reviews (up to 104K tokens per document), declassified paranormal case files from The Black Vault (processed from PDFs via Azure Document Intelligence), biomedical papers from the Biodex corpus, and 227 police misconduct records (average 12.5K tokens) containing nested structures and heterogeneous sources such as reports, transcripts, and medical notes [89]. IntelligentETL targets enterprise mixed-data scenarios, primarily PDFs from procurement workflows (invoices, purchase orders, receipts, contracts, and manuals) that embed both structured tables and free-text content, alongside supplementary CSV, XLSX, TXT files and relational sources loaded into SQLite [35].

Both systems rely on modern LLM-centric stacks for implementation. DocETL uses a declarative YAML DSL with Jinja2 templating, GPT-4o / GPT-4o-mini APIs for generation and validation agents, and auxiliary services such as Azure Document Intelligence and Nominatim for preprocessing and geocoding; the framework is fully open-source (github.com/ucbepic/docetl). IntelligentETL employs LangChain and LangGraph for multi-agent orchestration and persistent knowledge graph state, Chroma as the vector store for RAG, Tavily for web-context retrieval, pdfplumber + camelot for PDF/table extraction, pandas + SQLAlchemy for data handling, and a Gradio chatbot interface. These toolchains illustrate the shift from static ETL scripts to adaptive, agent-driven pipelines that integrate reasoning engines, vector databases, and specialized extraction libraries.

7.4 Future Directions

Despite promising accuracy gains and automation levels, several challenges remain for LLM-agent ETL systems. Standardized benchmarks that go beyond ELT-Bench to include real-time streaming, schema drift, self-healing under production loads, and multi-agent coordination across heterogeneous sources are urgently needed for reproducible progress. Hybrid human-agent collaboration frameworks will also be essential: agents excel at routine decomposition, extraction, and rewriting, yet high-stakes semantic decisions and edge-case mappings still require human oversight through intuitive visualization and approval interfaces.

8 LLM Agents for Text-to-SQL

Querying structured data has been a core challenge in database systems since the emergence of relational databases. Early systems required procedural programs, but the introduction of SQL shifted the paradigm to declarative querying—allowing users to specify what data they needed rather than how to retrieve it. As databases grew in scale and schema complexity, SQL queries became harder for non-experts to formulate, motivating the development of Text-to-SQL. Initial work explored mapping natural language questions to database queries through information extraction and conceptual query techniques [72]. Subsequent research adopted neural models to generate SQL from text [9], and the rise of generative AI accelerated progress by enabling LLM-based Text2SQL systems. This evolution paved the way for agentic approaches: unlike earlier one-shot models, these systems incorporate execution feedback and multi-step reasoning, improving robustness when handling complex schemas and ambiguous user intent [103].

8.1 Survey

Early single-LLM-agent approaches leverage execution feedback and structured reasoning. **ExeSQL** [123] implements a three-stage agentic loop in which the model first generates multiple candidate SQL queries tailored to different execution environments. The results are fed back directly to the LLM, enabling iterative refinement based on observed outcomes.

Recent work has adopted *multi-agent* architectures that decompose the task across specialized LLM agents working collaboratively. **MAC-SQL** [110] introduces three coordinated agents: a Selector Agent that prunes large schemas to retrieve only relevant tables, a Decomposer Agent that breaks complex user queries into simpler subtasks, and a Refiner Agent that uses execution feedback to iteratively correct faulty SQL. By distributing the cognitive load, MAC-SQL significantly outperforms single-agent baselines on benchmarks such as BIRD. **SQLFixAgent** [11] takes a different collaborative approach, with agents analyzing the same query from complementary perspectives: an SQLReviewer critiques semantic errors, a QueryCrafter suggests alternative formulations, and an SQLRefiner selects and integrates the best candidate. This multi-perspective validation reduces error propagation and enhances robustness on complex, multi-table queries. **MageSQL** [90] demonstrates a complete multi-agent pipeline for end-to-end Text-to-SQL applications, orchestrating multiple LLM agents in a practical, production-oriented framework.

Beyond collaboration, some systems emphasize trial-and-error refinement with strong reliability guarantees. **Intelli-Dispatch-SQL** [67] is designed for safety-critical domains such as power dispatching. It employs a layered agent architecture with an Intent Recognition module to parse high-level user goals and an SQL Validation module to ensure syntactic correctness and domain-specific semantic appropriateness before execution. Execution logs further support ongoing decision-making and traceability.

In enterprise settings, Text-to-SQL using intelligent agents + LLMs [70] embed LLM agents within broader analytics workflows. Their framework dynamically routes queries to the most

Table 6. AI Agents for Text to SQL [Table not referenced in text. Why 'Cleaning Tasks' column?]

Method	Agent Type	Orchestration	Planning	Memory	Prompting	SQL Generation Strategy	Data Types
ExeSQL [123]	Single agent	Execution-aware loop	Iterative refinement	Failure memory	Instruction-based prompts	Execution-driven bootstrapping via iterative query generation, execution-based filtering, and preference-based training	Relational
MAC-SQL [110]	Multi-agent	Collaborative coordination	Task decomposition	Shared context	Role-specific prompts	Question decomposition with optional database simplification, followed by execute-and-refine SQL generation	Large-scale Relational
SQLFixAgent [11]	Multi-agent	Consensus-based collaboration	Repair-oriented planning	Failure-aware memory	Critique-based prompts	Detect-and-repair SQL generation using semantic review, candidate repair generation, and final repair selection	Relational
MageSQL [90]	Multi-agent	Pipeline orchestration	Multi-stage planning	Shared monitoring state	Template-based prompts	Graph-based few-shot demonstration selection with post-generation error correction	Relational
Intelli-Dispatch-SQL [67]	Multi-agent	Modular orchestration	Intent-driven planning	Execution logs	Instruction prompts	Intent-recognition-guided SQL generation with SQL validation	Domain-specific Relational
Text-to-SQL using intelligent agents + LLMs [70]	Multi-agent	Agent-assisted workflow	Adaptive planning	Monitoring state	Instruction prompts	Agent-assisted natural-language-to-SQL translation, comparing domain-specific training with few-shot guidance	Enterprise Relational

suitable LLM (fine-tuned domain-specific or general-purpose) based on complexity, executes the generated SQL through controlled interfaces, and evaluates both correctness and performance. Emphasis is placed on workflow-level success, optimization effectiveness, policy compliance, and operational scalability.

Taken together, single-agent and multi-agent Text-to-SQL systems span complementary points in the design space, each balancing orchestration complexity, execution cost, and robustness differently. Current trends suggest a move toward hybrid architectures that pair efficient single-agent reasoning with targeted multi-agent validation and repair, enabling production-grade reliability without incurring full multi-agent overhead.

8.2 Metrics and Benchmark

LLM-based Text-to-SQL agents require multidimensional evaluation because they perform internal reasoning, iterative planning, and tool-driven decision-making before producing a final query. While SQL accuracy remains essential, these systems are assessed on broader criteria that capture execution grounding, robustness, and task-level success, reflecting their ability to decompose complex instructions and adapt during multi-step workflows. Execution-grounded agents such as ExeSQL are evaluated primarily on output accuracy, but their reliability is further measured through *in-distribution (ID)* and *out-of-distribution (OOD) generalization performance*, which indicates how well they handle queries that differ from their training examples.

Failure-aware agents such as **SQLFixAgent** evaluate performance through *repair effectiveness* and *execution success rate*, emphasizing not only their ability to generate correct SQL on the first attempt but also how efficiently and accurately they can recover from erroneous queries. In contrast, multi-agent systems like **MAC-SQL** *decompose complex tasks into smaller constituents* and assess performance using *task-level completion accuracy*. Domain-oriented agents such as **Intelli-Dispatch-SQL** prioritize *intent fulfillment*—capturing the user’s underlying question or objective even when key parameters are implicit or domain-specific—alongside *execution validation accuracy*. Because users often omit required details in prompts, Intelli-Dispatch-SQL is also evaluated on its ability to infer missing parameters correctly. Finally, enterprise-oriented workflows such as **Text-to-SQL using intelligent agents + LLMs [name?]** measure *workflow completion accuracy*, reflecting whether the system can execute an entire end-to-end analytical process, and *SQL optimization effectiveness*, which captures how well the system produces queries that are not only correct but also efficient and scalable.

Single-agent systems such as **ExeSQL** and **SQLFixAgent** primarily use *Spider* [121] as their core evaluation benchmark. In addition, ExeSQL incorporates *WikiSQL* [130] to provide complementary large-scale, single-table evaluation. A defining characteristic of ExeSQL is its multi-dialect evaluation via direct SQL execution, enabling assessment across heterogeneous database backends. To measure reasoning capabilities, ExeSQL includes *BIRD* [7], and to assess robustness under *distributional shifts*, e.g., differences between training conditions and real-world inference scenarios such as schema variation, question phrasing, query complexity, or domain semantics, it uses *Dr. Spider* [12]. Meanwhile, **Text-to-SQL using intelligent agents + LLMs [name?]** focus on enterprise-oriented evaluation settings.

Multi-agent systems such as **MAC-SQL** and **Intelli-Dispatch-SQL** are evaluated on domain-specific relational databases rather than standard academic benchmarks. These real-world databases are designed to capture practical domain constraints, schema heterogeneity, and operational requirements that academic datasets often fail to represent. Consequently, evaluation in these settings emphasizes task completion, intent fulfillment, and overall system-level effectiveness rather than exact structural query matching, aligning the assessment with realistic deployment needs.

8.3 Data and Tools

Agentic Text-to-SQL systems are predominantly trained on relational databases, each consisting of tables, column names, and foreign key relationships. The complexity of these databases varies widely, from simple single-table schemas to enterprise-scale designs. Most datasets are provided by users or sourced from academic repositories such as the Yale Semantic Parsing Group [28] and Salesforce Research [84]. To better approximate real-world conditions, these databases are intentionally made noisy, incorporating issues such as typos, naming inconsistencies, and incomplete schema labels, all of which help evaluate an agent’s robustness in practical environments.

A first group of systems such as **ExeSQL** and **SQLFixAgent** are trained on academic benchmark datasets including *Spider* [121], *BIRD* [122], *WikiSQL* [129], and *Dr. Spider* [13]. These datasets span a spectrum of noise levels, from the clean, single-table *WikiSQL* to the heavily perturbed *Dr. Spider*, which introduces synthetic noise to encourage robustness. As noted earlier, **ExeSQL** and **STaR-SQL** operate as single-agent systems. For execution, **ExeSQL** relies on SQL engines such as PostgreSQL, MySQL, and Oracle, whereas **STaR-SQL**, lacking a built-in execution module, depends on external execution sources. In contrast, **SQLFixAgent** functions as a multi-agent system and uses an execution validator to run and verify the SQL queries it generates.

The second group of systems covered in this survey are trained on domain-specific datasets. Both **MAC-SQL** and **Intelli-Dispatch-SQL** operate as multi-agent systems. **MAC-SQL** is trained on *BIRD*, while the datasets used to train **Intelli-Dispatch-SQL** have not been publicly disclosed.

The BIRD dataset is derived from real-world applications and therefore includes schema sprawl, incomplete metadata, and query ambiguity, enabling models to be stress-tested under realistic complexity. **MAC-SQL** incorporates a built-in SQL executor for running generated queries, whereas **Intelli-Dispatch-SQL** relies on an external execution engine.

Multi-agent systems such as **Text-to-SQL using intelligent agents + LLMs [name?]** are trained on enterprise or proprietary datasets. In particular, Text-to-SQL using intelligent agents + LLMs [name?] use an enterprise business-analytics database (proprietary, with 200 Q-SQL pairs for evaluation) as their training and testing environment. Both systems rely on external tools to execute the SQL queries they generate.

8.4 Future Direction

The future of LLM-based agentic systems in Text-to-SQL lies in building more general, robust, and context-aware models capable of handling the variability inherent in real-world environments. Although current systems achieve high accuracy on structured academic benchmarks, they often struggle with noisy schemas, domain constraints, and multi-turn interactions. Bridging this gap will require hybrid architectures that integrate lightweight orchestration with scalable multi-step reasoning, ensuring both efficiency and reliability. Incorporating external knowledge sources such as domain glossaries, historical query logs, and metadata will be essential for generating semantically precise queries in specialized domains. Progress also depends on developing benchmarks that better capture real-world schema ambiguity, natural language noise, and optimization objectives beyond correctness, including query efficiency, latency, and explainability. Ultimately, next-generation Text-to-SQL agents must be not only accurate but also adaptive, interpretable, and capable of supporting dependable natural language interfaces over dynamic, heterogeneous databases.

9 LLM Agents for SQL Query Optimization

SQL query rewriting involves transforming a database query into a semantically equivalent form that executes more efficiently. Traditional approaches rely on fixed rule-based optimizations or learned models that apply predefined rewrite rules, but these methods cover only a limited subset of query patterns and often struggle with complex or previously unseen structures. Recent advances in large language models introduce a new paradigm in which LLM-based agents can rewrite SQL beyond the constraints of static rules by leveraging their broader understanding of code and natural language. However, these agents must be carefully engineered to avoid producing incorrect or non-equivalent queries and to prevent performance regressions. Emerging systems explore how to combine LLM reasoning with tools, execution feedback, and validation mechanisms to achieve robust, automated SQL rewriting for query optimization.

9.1 Survey

SQL query optimization remains a difficult core task in database systems, as classic rule-based and cost-based optimizers handle familiar patterns well but falter on novel query shapes, complex joins, heterogeneous engines, and evolving workloads. LLM-powered agents address these gaps with autonomous reasoning, tool-assisted feedback (e.g., execution traces, plan simulation), multi-step planning, and self-correction. The following discussion focuses on systems that explicitly implement agentic behavior—multi-agent frameworks, FSM-orchestrated agents, ReAct-style loops, and code-agent architectures—rather than single-prompt LLM rewriting or non-agentic learned models.

Table 7. LLM-agent systems for SQL query optimization [table not referenced in text]

Method	Agent Type	Orchestration	Memory	Scalability	External Tools	Query Optimization Strategy	Query Type
QUITE [93]	Multi-Agent (FSM-controlled)	Finite State Machine + feedback loop	Agent memory buffer (rewrite history)	Broad query patterns	DB execution engine, rewrite middleware, hint injection	Agent-driven SQL rewriting with semantic-equivalence verification and execution-plan improvement via hint injection	Complex SQL (TPC-H, TPC-DS workloads)
GenDB [43]	Multi-Agent (Code Agent)	Multi-agent synthesis loop using Claude Code Agent	Not explicit in paper	OLAP-scale workloads	Claude Code Agent, execution sandbox, workload analyzer	LLM-driven synthesis of instance-optimized query execution code tailored to data, workload, and hardware	TPC-H + OLAP workloads

QUITE [93] is a training-free, feedback-aware multi-agent system for query rewriting beyond traditional rules. A finite state machine orchestrates specialized LLM agents that propose rewrites, invoke external tools for execution feedback, validate semantic equivalence, and inject hints to improve plans. It delivers up to 35.8% lower execution time and 24.1% more valid rewrites than rule-based or prior LLM baselines on complex workloads.

GenDB [43] introduces an LLM-powered agentic framework that moves beyond traditional engines entirely. Using a multi-agent architecture centered on Claude Code Agent, it analyzes schema, workload, and hardware, then synthesizes fully customized, instance-optimized execution code for each query. On TPC-H and custom OLAP benchmarks, it outperforms DuckDB, Umbra, MonetDB, ClickHouse, and PostgreSQL by significant margins.

These works mark the shift from static optimizers to autonomous, feedback-driven, and even code-synthesizing LLM agents.

9.2 Metrics and Benchmark

Evaluation of LLM-agent SQL optimizers blends classical database metrics with agent-specific measures of autonomy and reliability. Core performance indicators include execution time reduction, throughput, and resource utilization on standardized benchmarks (TPC-H, TPC-DS). Semantic equivalence (verified via execution results or query optimizer equivalence checks) and plan quality (cost estimates or actual runtime) are mandatory. QUITE reports up to 35.8% runtime improvement while preserving 100% equivalence on tested workloads. Agent-specific metrics capture rewrite coverage (percentage of queries successfully rewritten beyond rules), feedback-loop efficiency (number of iterations until convergence), and token/cost efficiency of the agentic process.

Traditional optimizers are evaluated with TPC benchmarks or real workloads, but these do not test agentic capabilities such as tool use, self-correction via execution feedback, or dynamic code synthesis. No dedicated end-to-end benchmark for LLM-agent query optimization exists yet; current evaluations rely on extended TPC suites plus custom enterprise workloads. QUITE and GenDB introduce their own task-specific suites (rewrite success + runtime on complex queries; code-generation correctness + end-to-end performance). Standardized benchmarks that systematically evaluate semantic safety, cost of agent loops, multi-engine interoperability, and robustness to schema drift remain an open challenge.

9.3 Data and Tools

QUITE primarily targets complex analytical SQL workloads drawn from TPC-H, TPC-DS, and real enterprise datasets on PostgreSQL and similar engines, integrating directly with the underlying DBMS to obtain real-time execution feedback and inject optimizer hints. Its multi-agent architecture orchestrates frontier LLMs (Claude-3.7-Sonnet, DeepSeek-R1, GPT-4o) through a finite-state

machine and custom middleware for rewrite generation and plan validation. **GenDB**, by contrast, focuses on OLAP workloads including TPC-H and a new leakage-resistant benchmark and synthesizes executable code tailored to specific data distributions, hardware characteristics, and optimization goals; its prototype combines a Claude Code Agent within a multi-agent loop, a workload analyzer, and an execution sandbox, producing code that can target any backend but is evaluated on leading columnar engines. Across these systems, common supporting components include LangGraph-style orchestration, database connectors for execution feedback, query-equivalence checkers, and frontier LLM APIs as the reasoning core, together enabling a shift from fixed, rule-driven optimizers to adaptive, self-improving agentic pipelines.

9.4 Future Directions

Several challenges remain for LLM-agent-based SQL optimization. Standardized benchmarks that jointly assess semantic safety, execution improvement, agent-loop cost, and robustness across engines and evolving workloads are still lacking. High-stakes production use will require hybrid human-agent collaboration, where agents propose and validate rewrites or synthesized code and humans provide final approval through visualized plan alternatives. Additional open issues include secure, auditable interoperability with legacy optimizers and cloud-native engines, governance agents that enforce compliance and cost constraints, and energy-aware scheduling of expensive agent loops. Extending today's predominantly batch and OLAP-focused systems to streaming, real-time, and hybrid transactional-analytical workloads with continuous self-optimization will be a critical step toward fully autonomous, instance-optimized query processing.

10 LLM Agents for Database Tuning

Database tuning is the process of optimizing a database so that it runs faster, efficiently and is more reliable. With the rise of Agentic AI researchers have moved away from traditional rule-based heuristic database tuning to leveraging the power of LLM based AI to tackle database tuning. A well-tuned database can handle, large volumes of data reliably, prevent bottlenecks such as slow queries and excessive disk usage and lower infrastructure cost by making better use of existing hardware

10.1 Survey

AI agents powered by large language models (LLMs) can optimize configuration parameters, indexes, and queries. These agents plan multi-step processes, leverage external tools to achieve goals, and refine their approaches based on feedback. They execute queries, consult manuals, and retain memory of past actions to iteratively improve performance.

A detailed comparison of representative systems from 2023–2025 is provided in Table 8, which highlights key architectural and functional differences across agent type, planning strategy, memory usage, prompting techniques, tuning focus, and supported databases.

The first category we discuss is *knob configuration optimization agents*. These agents identify optimal settings for databases with hundreds of tunable parameters. Traditional ML-based tuners treat this as a black-box problem solved via trial-and-error. In contrast, modern LLM-based agents incorporate domain knowledge for more efficient exploration.

AgentTune [48] employs a multi-agent system with four specialized LLM-based agents: (1) a Workload Analyzer that inspects SQL workloads and system status to detect patterns (e.g., query complexity, data volumes); (2) a Knob Selector that identifies the most impactful knobs for the workload; (3) a Range Pruner that defines safe and effective value ranges (avoiding invalid settings like excessive memory allocation); and (4) a Configuration Recommender that iteratively proposes and refines configurations using GPT-4 in a loop. It incorporates DBMS feedback (performance

metrics), employs tree-structured search, uses semantic reasoning, centroid-based ranking of proposals, and retains memory of prior iterations for feedback-driven refinement.

Extending configuration tuning to the storage layer, **StorageXTuner** [52] is an LLM agent-driven framework for automatic tuning of heterogeneous storage engines (e.g., RocksDB, LevelDB, CacheLib, MySQL InnoDB). It decomposes the process across four specialized agents: Executor (handles sandboxed benchmarking), Extractor (creates performance digests via LLM-generated parsers), Searcher (performs insight-guided tree search over configurations), and Reflector (generates, manages, and validates insights in layered memory: short-term for tentative, long-term for verified). This modular, insight-driven design enables efficient, safe, and generalizable tuning across diverse storage systems and workloads (e.g., YCSB, TPC-H/C), achieving substantial throughput gains (up to 575% over defaults) with fewer trials.

The next category is *index selection agents*, which support physical database design by recommending indexes to accelerate query workloads. Traditional methods rely on combinatorial search or heuristics over vast spaces.

AMAZe [49] introduces a zero-shot, multi-agent LLM framework for index advising. It decomposes the task into steps mimicking a human DBA: a high-level coordinator agent oversees the process, while specialized agents collaborate in a hierarchical worker-manager style. By dividing responsibilities and using chain-of-thought reasoning with cost estimation, AMAZe explores the design space more intelligently than single-pass prompting, outperforming both traditional heuristics and prior LLM-based methods.

Another application involves workload analysis and problem diagnosis via autonomous database co-pilots.

Panda [92] emphasizes context grounding over pure LLM generation. It features four cooperating agents/modules: (1) Grounding, which collects relevant data (metrics, logs, knobs); (2) Verification, which checks suggestion feasibility (e.g., valid indexes or knob ranges); (3) Affordance, which evaluates risks and applicability in the user’s context; and (4) Feedback, which logs outcomes (successes, side effects) to enable learning over time.

Finally, systems like **AskDB** [78] enable natural language interaction with relational databases, supporting both analytical queries and administrative operations (including configuration and tuning-related tasks) via schema-aware prompting, multi-step task decomposition, and execution on models like Gemini. While primarily focused on NL-to-SQL and admin commands, it complements tuning agents by providing an accessible interface for initiating optimization workflows.

Table 8. LLM Agents for database tuning

System	Agent Type	Architecture	Planning	Memory	Orchestration	Tuning Focus	Database
AgentTune [48]	Multi-agent	Sequential agents	Tree-search refinement	Iteration-level feedback	Role-based agent coordination with validator agents	Knob configuration	PostgreSQL, MySQL, TiDB
AMAZe [49]	Multi-agent	Hierarchical	Decomposed planning stages	Chain-of-thought state	Hierarchical coordination between planner and tuning agents	Index recommendation	PostgreSQL, MySQL
Panda [92]	Single agent	LLM + modules	Reactive validation loop	Issue context memory	Module-based execution pipeline driven by the LLM	Performance diagnosis	Amazon Aurora (MySQL/Postgres)
AskDB [78]	Single agent	LLM + schema-aware prompting + task decomposition	Task decomposition for multi-step actions	Schema/metadata + interaction context	Task-decomposition pipeline coordinating query analysis and DBA actions	DBA assistance and multi-step database management actions	Relational SQL databases (benchmarks + DBA tasks)

10.2 Metrics and Benchmarks for Performance Measurement

Performance optimization using LLM-powered agents focuses not only on final tuning outcomes but also on the efficiency, convergence speed, safety, and robustness of the tuning process itself. Standard database benchmarks remain central for evaluating end-to-end performance improvements.

Commonly used workloads include OLTP-oriented benchmarks such as **TPC-C** [99] and **YCSB** [19], as well as analytical benchmarks like **TPC-H** [100] and **TPC-DS** [101]. Systems like GPTuner and AgentTune additionally assess tuning *efficiency* through metrics such as time-to-optimal configuration, number of replayed workloads (or trials), cumulative regret, and composite efficiency scores that balance performance gains against tuning overhead, enabling fair comparisons across long-running experiments and varying workloads.

For knob and storage tuning agents:

- **AgentTune** [48] evaluates stability under workload shifts, pruning of invalid configurations, and overall performance gains on PostgreSQL, MySQL, and TiDB.
- **λ -Tune** [27] measures end-to-end throughput/latency after one-shot or few-trial execution on PostgreSQL and MySQL.
- **StorageXTuner** [52] demonstrates substantial gains on heterogeneous storage engines (RocksDB, LevelDB, CacheLib, MySQL InnoDB) using YCSB, MixGraph, TPC-C, and TPC-H/C workloads. It achieves up to 575% throughput improvement and 88% p99 latency reduction over baselines, with additional metrics like MPG (mean performance gain), TC95 (trial cost at 95% performance), TE (tuning efficiency), and TWER (tuning workload execution ratio) to quantify convergence speed and robustness with fewer trials.

For index recommendation:

- **AMAZe** [49] evaluates benefit-to-cost ratio, query execution time reduction, and index creation overhead on PostgreSQL using benchmarks such as TPC-H, JOB (Join Order Benchmark) [44], and real-world datasets. It emphasizes zero-shot performance across varying budget constraints (e.g., 10%–50% index sets) and outperforms heuristics in most settings.

Quality, safety, and robustness of agent decisions are critical, particularly to avoid redundancy, unsafe operations (e.g., out-of-range knobs, crash-inducing settings, invalid SQL/index suggestions), or side effects.

Diagnosis and co-pilot agents prioritize accuracy and utility:

- **Panda** [92] assesses recommendation usefulness via grounding/verification/affordance/feedback loops, using real telemetry (e.g., 250+ metrics over days) from Amazon Aurora (MySQL/PostgreSQL) to measure issue resolution rate and reduction in vague/generic suggestions.

General interaction agents like **AskDB** [78] use execution accuracy (execution-equivalent results) on Text-to-SQL benchmarks such as Spider 1.0 and Spider 2.0 subsets, achieving high scores (e.g., 89.8% on diverse Spider subsets) with low interaction turns (average 1.34–1.45), plus DBA-task success rates for administrative operations.

Traditional benchmarks (TPC-C/H/DS, YCSB, JOB) excel at measuring workload performance but fall short in assessing agent-specific traits: adaptation speed to workload changes, long-term autonomous operation, robustness to noise/errors, safety under constraints, and convergence efficiency. As LLM agents become more autonomous and iterative, dedicated agent-centric benchmarks—incorporating dynamic workloads, safety constraints, multi-turn interaction costs, and online adaptation—are needed to fully evaluate their practical value in real database environments.

10.3 Data and Tools

To assess the performance of LLM-based agents for database tuning, appropriate benchmarks, workload representations, and integrated toolchains are essential. Standard database benchmarks, while reliable for measuring end-to-end performance, often fail to fully replicate real-world conditions such as dynamic, long-running, or noisy production environments. As a result, many LLM agent designs incorporate workload summarization or structured feature extraction pipelines rather than feeding raw, voluminous query logs directly to the model. This reduces input complexity while retaining the essential characteristics needed for effective reasoning and decision-making.

Workload handling varies across systems. For instance, AgentTune extracts structured features from SQL workloads, including query frequencies, join patterns, data access distributions, query complexity indicators, and current system status metrics. These features inform its specialized agents, such as the Workload Analyzer and Knob Selector, enabling targeted knob identification. In contrast, λ -Tune relies on large-context prompting to incorporate a comprehensive description of the entire workload and DBMS context in a single input, allowing the LLM to reason directly over the full setting and generate complete configuration scripts without intermediate reduction steps. Diagnostic agents like Panda and Andromeda take a different approach by grounding decisions in heterogeneous, rich data sources—such as system metrics, query execution logs, telemetry time series, and incident tickets—rather than depending solely on summarized query representations.

The choice of underlying language models also differs. Most surveyed systems, including GPTuner, AgentTune, Panda, and StorageXTuner, directly utilize powerful pre-trained general-purpose LLMs (most commonly GPT-4 or equivalent models) for domain knowledge extraction, multi-step planning, configuration proposal, and iterative refinement. Andromeda stands out by training a domain-specific retrieval model using contrastive learning. This model produces unified embeddings for a diverse corpus that includes DBMS manuals, Stack Overflow-style Q&A posts, production incident logs, knob-related questions, and synthetically augmented data, enabling more accurate and contextually relevant retrieval-augmented generation tailored specifically to configuration debugging tasks.

Toolchains for agentic database tuning typically combine a DBMS client interface, workload execution harness, monitoring components, and safety mechanisms to support safe and reproducible interaction. Benchmarking harnesses such as Sysbench or OLTP-Bench (and custom scripts in many cases) are commonly used to reset configurations, execute standardized or representative workloads (e.g., TPC-C, YCSB), and collect key performance metrics like throughput, latency, and resource consumption. AgentTune, for example, employs a central controller that iteratively runs SQL scripts, parses execution results, and returns performance feedback to guide subsequent agent refinements.

To mitigate the high cost of repeated real executions, some systems employ surrogate models. AMAZe, in particular, leverages the native cost estimates provided by the PostgreSQL or MySQL query optimizer as a fast proxy for actual query execution time during hierarchical agent planning and index benefit evaluation, significantly accelerating exploration in the large search space of index combinations.

Diagnostic agents require additional specialized tools. Andromeda, for instance, includes components for parsing and analyzing time-series telemetry data to detect anomalies, then translates those anomalies into concise natural language descriptions that the LLM can reason over alongside retrieved documents. Many designs also incorporate sandboxed benchmarking environments (as seen in StorageXTuner) to prevent system crashes or data corruption from invalid or dangerous configuration proposals.

In summary, the toolchains supporting LLM-driven database agents are increasingly standardized around classical benchmarks augmented with realistic workloads, domain-specific knowledge injection (through manuals, RAG, or carefully engineered prompts), and robust APIs/interfaces that enable controlled, safe, and feedback-rich interaction with the DBMS, its optimizer, monitoring systems, and execution environment. This ecosystem supports reproducible evaluation, minimizes operational risks, and facilitates truly autonomous, iterative tuning in environments that closely mirror production use cases.

10.4 Future Direction

AI agents are very powerful and are capable of automating many of the mundane tasks but, suffer from hallucinations as a result of which many researchers don't consider them to be trustworthy. Many of the agentic systems discussed here is trained based upon traditional databases which fails to capture the multi-dimensional benchmark required for LLM Agents. For multi-agent systems there is a lack of benchmarks as to how well they are cooperating amongst themselves and how quickly one agent can fix the mistake of other agent and how efficiently they can coordinate with one another.

11 Fact Checking with LLM Agents

In the era of social media and rapid information dissemination, the digital world is saturated with misinformation, making fact-checking not merely a convenience but an essential safeguard for informed discourse. Traditionally performed manually by journalists and experts, fact-checking has become impractical due to the sheer volume and velocity of online content. The advent of large language models (LLMs) and LLM-based agents has shifted the paradigm toward automated, scalable verification. These agents perform multi-step planning, leverage external tools for evidence gathering, and refine outcomes through feedback loops. They decompose complex claims, retrieve and assess heterogeneous evidence, manage uncertainty, and dynamically adapt strategies—moving beyond static classification pipelines to collaborative, reasoning-driven processes.

11.1 Survey

The emergence of LLMs has inspired a wave of research on automated fact-checking, evolving from single-model entailment to iterative, multi-agent collaborative reasoning. Early approaches framed fact-checking as a distributed inference problem, where noisy individual agent judgments are aggregated for improved collective accuracy. Verma et al. [106–108] pioneered this direction with multi-agent architectures that model fact-checking as group decision-making, employing adaptive consensus algorithms and dynamic reweighting of agent contributions to handle non-stationary conditions (e.g., evolving claims or agent behaviors).

These estimator-driven frameworks laid the foundation for more sophisticated systems. **Distributed Fact Checking** [106] introduced the core idea, later extended in **Multi-Agent Fact-Checker: Adaptive Estimators** [107], which enables internet connectivity for real-time evidence-based judgments.

With advances in LLM reasoning, agentic pipelines emerged that actively search for evidence, debunk claims, and verify veracity through iterative tool use. **FactAgent** [47] exemplifies this trend: users input a URL, and the agent searches the web iteratively, reasons over retrieved evidence, and outputs judgments—shifting from passive models to execution-aware agents.

Pipeline-style multi-agent architectures decompose tasks into specialized roles (e.g., retrieval, reasoning, critique, verdict synthesis). Notable examples include **LoCal** [60] (focused on logical/causal consistency via structured evidence graphs), **DelphiAgent** [116] (emphasizing iterative critique and self-correction), **FIRE** [115], and **MARO** [45] (disentangling reasoning and evaluation

for transparency). **Multi-agent Systems for Misinformation Lifecycle** [26] extends this to the full misinformation cycle: classification, detection, correction, and source identification via five specialized agents.

Debate- and consensus-driven approaches further enhance robustness by having agents argue, critique assumptions, and converge on verdicts. **LLM-Consensus** [41] iteratively discusses claims to resolve disagreements with justifications. **MAD-Sherlock** [42] uses assumption-based debate for contextual/multimodal disinformation, seeking disproof and external evidence. **TruEDebate** [56] formalizes adversarial debate with opposing agent sides, multiple rounds of arguments/counterarguments, and synthesis for logical consistency.

Recent enhancements include **GKMAD** [62], which injects structured guidance and external knowledge into debates for more targeted, informative discussions and reliable verification. Verifiable multi-tool frameworks like **Toward Verifiable Misinformation Detection** [21] integrate precise search, credibility assessment, and numerical verification tools with traceable evidence logs.

A comprehensive comparison of representative agent-based fact-checking systems is provided in Table 9.

Table 9. LLM Agents for fact checking

Method	Agent Type	Orchestration	Planning	Memory	External Tools	Verification Strategy	Data Types
FactAgent [47]	Single agent	Iterative execution loop	Task decomposition	Short-term context	Web search, scraping	Evidence-based claim verification	Web text
LoCal [60]	Multi-agent	Pipeline orchestration	Logical / causal planning	Intermediate reasoning state	Retrieval, reasoning modules	Logical and causal consistency checking	Textual evidence
FIRE [115]	Single agent	Iterative retrieval loop	Step-wise refinement	Execution trace	Web search	Evidence accumulation and validation	Web text
MARO [45]	Multi-agent	Expert collaboration	Role-driven planning	Shared reports	Search, analysis tools	Cross-domain misinformation detection	News text
DelphiAgent [116]	Multi-agent	Deliberative coordination	Critique-driven planning	Argument history	Retrieval tools	Iterative evidence critique	Textual evidence
MAD-Sherlock [42]	Multi-agent	Debate-based orchestration	Contextual reasoning	Debate history	Web search	Contextual misinformation detection	Image-text
TruEDebate [56]	Multi-agent	Adversarial debate	Argumentative planning	Debate summaries	Retrieval tools	Consensus through structured debate	Textual claims
GKMAD [62]	Multi-agent	Guided debate	Knowledge-guided planning	Debate + knowledge state	External knowledge retrieval	Guided & knowledgeable debate verification	Textual/multimodal
Verifiable Misinfo Detection [21]	Single/Multi-agent	Tool-augmented loop	Evidence traceability planning	Evidence logs	Multi-tool (search, credibility, numerical)	Verifiable evidence-based detection	Web text/numerical
Misinfo Lifecycle [26]	Multi-agent	Lifecycle pipeline	Sequential role planning	Shared lifecycle state	Detection/correction tools	Full lifecycle (detect, correct, source ID)	Text/multimodal
FACT-AUDIT [51]	Multi-agent	Adaptive evaluation loop	Dynamic test generation	Model-specific responses	Importance sampling agents	Adaptive justification-aware evaluation	LLM responses

11.2 Metrics and Benchmark

Evaluating fact-checking agents is challenging due to diverse architectures, reasoning depths, and nuanced outputs (e.g., justifications, debates). Beyond binary accuracy, metrics must capture reasoning quality, evidence strength, robustness to noise, and debate dynamics.

Core metrics include precision, recall, and F1-score on claim veracity, often imbalanced toward false claims. For distributed/estimator frameworks (Verma et al.), estimation error and convergence rate assess aggregation quality. Debate-driven systems add consensus formation time, disagreement reduction, argument consistency, and healthy debate quality.

Evidence-centric agents (FactAgent, LoCal, Verifiable Misinfo Detection [21]) evaluate retrieved document relevance, pertinence, and credibility. Justification-focused metrics from **FACT-AUDIT** [51] include Insight Mastery Rate (IMR: proportion of low-scoring responses), Justification Flaw Rate (JFR), and overall Grade—probing verdict accuracy and explanation quality dynamically.

All systems report latency, computational cost, and hallucination rates. Benchmarks repurpose single-model datasets like **FEVER** [97], **LIAR** [112], **PolitiFact** [79], **Fakeddit** [66], and multimodal sets (**NewsCLIPPings**, **VERITE**, **MMFakeBench**). Open-world systems use **HotpotQA** [119] and **AVeriTeC** [88] for multi-hop/web evidence. Newer sets like **RealFactBench** [117] test temporal grounding.

No dedicated agentic benchmark exists yet—most reuse static datasets without accounting for multi-step reasoning, tool coordination, or debate protocols—hindering fair comparisons.

11.3 Data and Tools

Agentic fact-checking operates in open, heterogeneous environments mirroring real-world verification. Agents dynamically retrieve, generate, and reason over evidence from web crawls, intermediate states, and prior judgments—beyond fixed knowledge bases.

Common static benchmarks for testing include **FEVER** [97], **FEVEROUS** [3], **SciFact** [109], and **HealthVer** [87]. Open-world systems (FactAgent, FIRE, MARO, Verifiable Misinfo Detection) rely on live web content (news, pages) as noisy evidence. Multimodal agents (MAD-Sherlock, Misinfo Lifecycle) use **NewsCLIPPings** [59], **VERITE** [74], and **MMFakeBench** [54] for image-text coherence.

Estimator-based systems (Verma et al.) abstract data as labeled agent response sequences (noisy votes).

Tools primarily involve search engines, web scrapers, text summarizers/normalizers, and custom controllers. LLM reasoning decomposes tasks, triggers tools, and aggregates results. Pipeline systems encode roles architecturally; debate systems control via protocols; estimator systems use statistical aggregation. Verifiable frameworks add specialized tools (credibility scorers, numerical verifiers) with traceability.

In summary, agentic fact-checking prioritizes runtime flexibility over fixed pipelines, enabling realistic open-world operation but raising challenges in evidence credibility, tool reliability, and reproducibility—areas needing future standardization.

11.4 Future Directions

Despite advances, agent-based fact-checking remains an active research frontier. First, dedicated evaluation frameworks are essential: current datasets ignore multi-step stability, tool coordination, debate dynamics, uncertainty, and temporal drift. Adaptive benchmarks (inspired by FACT-AUDIT) should support open-world dynamics and cross-architecture comparison.

Second, robustness in noisy/adversarial settings requires improvement. Web-retrieved evidence risks noise, bias, correlated hallucinations, or cascading errors. Agents need structured knowledge graph constraints, confidence estimation, self-monitoring, and adversarial training.

Third, efficiency bottlenecks from multi-round debates and tool calls demand step-control techniques, early stopping, or lightweight agents.

Fourth, standardization of orchestration (task allocation, interaction, tool protocols) would enable reproducibility and fair comparison—perhaps via declarative specifications or shared interfaces.

Finally, hybrid approaches combining LLM agentic flexibility with estimator-based theoretical guarantees (e.g., statistical aggregation of agent judgments) remain underexplored and hold promise for reliable, provably sound verification.

12 Multi-Modal Data Management using LLM Agents

Today, there are often multiple types of data that are stored together in a data lake, such as tables, text, images, documents, etc. These data are often not joined and do not share a common schema. It is difficult for a database system to easily access data across all these different modalities for two

reasons: (1) a fixed query language is often not convenient for all types of data and (2) different data often requires different processing. A LLM agent is a multi-modal data management framework that can accept a natural language input and then call multiple existing tools to explore, search, and analyze data from multiple modalities. The LLM agent can perform multi-step planning to decompose a user’s intent into multiple modality-dependent sub-problems, call multiple tools to solve each sub-problem in parallel (including tools for structured data and tools for perceptual data), and then combine the results to generate a unified answer. This is different from a fixed ETL pipeline that only works for data from a fixed number of sources and/or a fixed query processor that only works for data of a single modality. The LLM agent can adjust its planning strategy based on what data are available, what tools are available, what is the cost of calling a tool, and even based on the intermediate results.

12.1 Survey

Early approaches to multi-modal data access treated different modalities as isolated silos requiring manual integration. The advent of LLM agents has enabled *unified multi-modal exploration*, where agents act as orchestrators coordinating specialized tools across modalities. These systems can be categorized by their orchestration architectures: *sequential query planners*, *DAG-based parallel explorers*, *cross-modal integration agents*, *declarative optimization systems*, *retrieval-augmented agents*, and *visual analytics systems* Table 10 covers all systems discussed here.

DAG-based parallel explorers address the efficiency limitations of sequential planning. M2EX [69] introduces LLM agents that decompose natural language queries into Directed Acyclic Graphs (DAGs) of sub-tasks, where nodes represent modality-specific operations (e.g., Text2SQL for structured data, VisualQA for images) and edges represent data dependencies. This architecture enables parallel execution of independent branches—querying structured databases while simultaneously analyzing images—and provides selective re-planning capabilities that rewire only affected sub-graphs when errors occur. M2EX demonstrates 30–42% accuracy improvements and 51% lower latency compared to sequential approaches by exploiting parallelism in multi-modal workflows. The predecessor system XMODE [68] established the foundational DAG workflow decomposition with emphasis on explainability, while M2EX extends this with performance optimization and dynamic re-planning.

Cross-modal integration agents tackle the challenge of unified analysis across structured and unstructured “zombie data”—visual documents and scanned reports that resist traditional querying. DataCross [81] introduces a divide-and-conquer architecture where specialized sub-agents perform deep exploration within individual modalities (SQL for structured data, vision models for documents) followed by contextual cross-pollination, where agents reinterpret sources in the context of others. The system employs a recursive ReAct (reReAct) mechanism for code generation and debugging, achieving 29.7% improvement in factuality over baseline approaches by enabling agents to recognize when visual documents supplement or contradict structured database entries.

Retrieval-augmented multi-modal agents focus on evidence gathering across heterogeneous sources. HM-RAG [53] employs a hierarchical multi-agent architecture where decomposition agents rewrite queries for different modalities, parallel retrieval agents gather evidence from vector databases and structured stores, and consensus mechanisms resolve discrepancies between sources. The system implements collaborative intelligence through consistency voting and expert model refinement, achieving 12.95% improvement over baseline RAG systems on multi-modal question answering benchmarks by explicitly handling conflicts between retrieved text and visual evidence.

Visual analytics agents specialize in interactive data exploration through visualization generation. LightVA [128] employs a Planner-Executor-Controller architecture where the agent recursively

decomposes analytical tasks, determines appropriate visualization types, and incorporates human feedback for refinement. The system distinguishes itself from static visualization tools by maintaining a task state stack that enables hierarchical planning—breaking complex analytical goals into sub-tasks that may involve querying structured data while simultaneously requesting user clarification on visual preferences.

Table 10. LLM Agents for Multi-modal data management

Method	Agent Type	Orchestration	Planning	Memory	External Tools	Multi-Modal Strategy	Data Types
XMODE [68]	Single agent	DAG-based agentic	Dynamic composition re-planning	Execution state + explanation cache	SQL engine, VisualQA, Python UDFs	Explainable parallel/sequential sub-tasks	Tables, text, images
M2EX [69]	Single agent	DAG-based parallel	Dependency-aware DAG	Execution graph state	Text2SQL, VisualQA, Python UDFs	Parallel sub-task execution	Tables, text, images
DataCross [81]	Multi-agent	Divide-and-conquer	Intra-source cross-modal	Shared embedding space	SQL, vision models, web search	Contextual cross-pollination	Structured + visual docs
HM-RAG [53]	Multi-agent	Hierarchical	Query decomposition	Retrieval cache	Vector DB, web search, graph DB	Consensus-driven retrieval	Text, images, structured
LightVA [128]	Single agent	Hierarchical	Recursive decomposition	Task state stack	Viz libraries, analytics tools	Visual analytics pipeline	Structured + visualization

12.2 Metrics and Benchmark

Performance optimization LLM Agents also concentrated on optimizing the efficiency and convergence of the tuning process. Performance was measured with standard OLTP benchmarks like **TPC-C** [99] and **YCSB** [19], and analytical benchmarks such as **TPC-H** [100] and **TPC-DS** [101]. GPTuner and AgentTune considered efficiency as well by using time-to-optimal, number of replayed workloads, cumulative regret, and a composite efficiency score that balances performance and tuning time, allowing for apples-to-apples comparison across long-term tuning runs and different workloads.

In addition to correct outputs, the quality, safety and robustness of the decision-making of these LLM agents are also measured. For LLM agents, we list the redundancy and unsafe database operations, e.g., out-of-range knob settings, config-induced system crash, and invalid SQL rewrites. GPTuner is measured by generating textbfvalid knob configurations to handle benchmark workloads while AgentTune further evaluates the textbfpruning of invalid configurations and stability in tuning performance when workload changes. For diagnosis LLM agents like Andromeda, textbfaccuracy-based metrics such as precision, recall and fix success rate are measured over real incident data crawled from production logs, manuals and tickets. In contrast Andromedia is measured using textbfaccuracy-based metrics such as precision, recall etc. The decision made by LLM Agents are also measured to ensure that these decisions are qualitative. Although many benchmarks like TPC-C [99], TPC-H [100], TPC-DS [101], YCSB [19] and Join Order Benchmark [44] have been widely used, these benchmarks do not consider how fast the agent adapts to the new workloads, how long the agent can run independently, and how robust the agent is. As a result, benchmark for AI agents should be adopted.

12.3 Data and Tools

To assess the performance of LLM-based agents, we need proper benchmarks, workload representations, and toolchains. Unfortunately, the standard databases for LLM Agents do not simulate real-world conditions. Instead of feeding full query logs to LLMs, some designs adopt workload summarization pipelines.

The systems surveyed in this paper, such as Panda, GPTuner, and AgentTune, utilizes pre-trained LLMs (e.g., GPT-4) except for Andromedia. Andromedia trains a domain-specific retrieval model

using contrastive learning to embed various types of documents, including manual, Q&A posts, incident logs, and knob-related questions generated for data augmentation.

Toolchain. Most agents are coupled with a DBMS client and a benchmarking tool in order to configure the DBMS, run the workload, and gather performance metrics. The system deploys a script or a benchmarking tool like Sysbench [39] or OLTP-Bench [22] in order to automatically reset configurations, run the workload, and extract the performance metrics. AgentTune employs a controller in charge of running a set of SQL scripts and parsing the results after each tuning iteration. To cut down the time needed for execution, some systems employ the cost model of the DBMS optimizer as a surrogate for the actual execution time. For example, this surrogate is used for index tuning or for the agent in AMAZe. Diagnostic tools usually require additional tools like, e.g., a log parser, a telemetry data analyzer. In Andromeda, e.g., the system analyzes time series data from system metrics in order to identify anomalies and describe them in a natural language for the LLM to reason about. In summary, the tools used for agentic DB tuning are converging towards classical benchmarks with real workloads, domain knowledge, and APIs that enable the LLM to interact with the DBMS and its monitoring in a safe and reproducible manner.

12.4 Future Directions

AI agents are very powerful and are capable of automating many of the mundane tasks but, suffer from hallucinations as a result of which many researchers don't consider them to be trustworthy. Many of the agentic systems discussed here is trained based upon traditional databases which fails to capture the multi-dimensional benchmark required for LLM Agents. For multi-agent systems there is a lack of benchmarks as to how well they are cooperating amongst themselves and how quickly one agent can fix the mistake of other agent and how efficiently they can coordinate with one another.

13 LLM Agents for Data Governance & Cataloging

Large Language Models (LLMs) are transforming the landscape of enterprise data governance. Traditionally, governance and cataloging tasks relied on brittle, rule-based systems that struggled to scale with data complexity and volume. Recent advances in LLMs offer a new paradigm: intelligent agents capable of interpreting natural language policies, reasoning over heterogeneous data sources, and executing governance workflows autonomously. These LLM agents are being deployed across the data lifecycle—from metadata enrichment and schema alignment to lineage tracking, quality assurance, and policy enforcement—enhancing efficiency, consistency, and compliance in modern data ecosystems.

13.1 Survey

Recent advancements have shifted toward ****LLM-powered agents**** and multi-agent systems for automating and monitoring data governance processes, including quality assurance, compliance enforcement, and interaction with data catalogs or marketplaces.

One prominent benchmark in this space is **DataGovBench** [57], which evaluates LLM agents on 150 diverse, real-world data governance tasks (e.g., filtering, imputation, deduplication, integration, classification, and compliance-related operations) derived from actual enterprise cases. The associated **DataGovAgent** framework employs a multi-agent architecture with Planner–Executor–Evaluator components: the Planner interprets natural-language governance intents and policies into constraint-guided Directed Acyclic Graphs (DAGs) using contracts and pre/post-conditions for compliance monitoring; the Executor generates grounded code via retrieval-augmented generation (RAG); and the Evaluator performs sandboxed execution, feedback-driven debugging, and validation. Experimental results demonstrate significant improvements, boosting the Average Task

Score (ATS) on complex multi-step pipelines from 39.7 to 54.9 while reducing debugging effort by 78%. These findings underscore the potential of agentic LLM systems as robust, context-aware tools for enforcing data quality standards and governance workflows, addressing limitations in single-model approaches that struggle with multi-step reasoning and error correction.

Complementing workflow automation, agentic systems have also been applied to semantic discovery, access control, and real-time governance in enterprise data catalogs. **Data Product MCP** [98] introduces an integration of the Model Context Protocol (MCP) into a data product marketplace, enabling LLM-powered agents to semantically discover, request access to, and query enterprise data products while enforcing governance constraints. Key features include tools for Data Product Search (metadata and business-context semantic search), Data Product Get, Request Access, and Query execution, with a Governance AI component (powered by LLMs such as GPT-4o) that validates requests and queries against data contracts, global policies, purpose limitations, and compliance rules in real time—blocking violations, issuing warnings, and generating audit trails. By bridging agentic AI workflows with federated data catalogs/marketplaces (compatible with platforms like Snowflake and Databricks), this approach maintains strong governance without manual interventions or workflow interruptions. Qualitative evaluations with data governance experts highlight its effectiveness in reducing technical barriers for analysis while preserving compliance.

These agent-centric contributions build on earlier LLM applications in governance, such as metadata augmentation [?], schema matching [1, 75], provenance extraction [31], and policy reasoning [24, 55], but extend them toward autonomous, multi-step monitoring and interaction in governed data ecosystems.

Table 11 summarizes representative LLM-based tools and frameworks for data governance, highlighting their primary focus areas and key capabilities.

Table 11. LLM Agents for data governance

Tool/Framework	Focus Area	Agent Type	Planning & Orchestration	Governance Features	Catalog/Marketplace Integration	Evaluation
DataGovAgent [57]	Data Governance Workflows	Multi-agent	Planner-Executor-Evaluator with constraint-guided DAGs from natural-language intents	Contracts, pre/post-conditions, compliance monitoring in pipelines	Indirect via schema/RAG grounding (workflow-level)	Quantitative benchmark (ATS 39.7 → 54.9, TSR +14pp, debugging effort ~78%)
Data Product MCP [98]	Data Catalog & Marketplace Governance	LLM-powered tool-calling agents	Natural-language intents mapped to tool calls (Search/Get/Request/Query)	Real-time Governance AI (policy/contract/purpose validation, violation blocking, audit trails)	Native semantic discovery, access requests, and governed querying in enterprise marketplace	Qualitative expert evaluation (n=16 scenarios and demos)

13.2 Metrics and Benchmark

Benchmarking efforts such as [57] are critical to understanding the limitations and progress of LLM agents in real-world data governance applications. DataGovBench comprises 150 diverse, enterprise-grounded tasks spanning core governance operations—including filtering, refinement, imputation, deduplication, integration, classification/labeling, and compliance-related transformations—designed to mirror practical challenges across multiple domains and data scales. These tasks range from atomic, single-operator actions to complex multi-step pipelines (DAGs) that require chaining operations, reasoning over constraints, and ensuring end-to-end compliance and quality.

The benchmark employs rigorous, automated evaluation metrics to assess pipeline reliability: **Task Success Rate (TSR)** measures the proportion of tasks where business objectives are fully

achieved; **Code Runnable Rate (CRR)** evaluates executable code generation; and **Average Task Score (ATS)** provides a normalized, composite score aggregating quality and correctness against ground-truth outcomes.

Results reveal that while general-purpose LLM baselines perform reasonably on isolated or simple tasks, their performance degrades markedly on multi-step workflows involving dependent constraints, noisy data, or iterative refinement—often due to insufficient planning, hallucinated code, or unhandled errors. The proposed DataGovAgent framework, with its Planner–Executor–Evaluator multi-agent architecture (constraint-guided planning, RAG-based code generation, and sandboxed feedback loops), delivers substantial gains: it boosts ATS on complex tasks from 39.7 to 54.9, increases TSR to 64% on operator-level tasks (+15 percentage points) and 60% on DAG-level tasks (+14 percentage points) with strong models like GPT-5, and reduces Average Debug Iterations (ADI) by over 77.9% (e.g., from 14.89 to 3.29 compared to baselines like ChatDev). These improvements highlight the value of iterative evaluation, structured agent decomposition, and retrieval augmentation in enhancing consistency, reasoning depth, robustness, and efficiency—key requirements for deploying trustworthy LLM agents in production data governance and cataloging scenarios.

Complementary systems like Data Product MCP [98] further validate these insights through qualitative expert assessments, demonstrating effective real-time governance enforcement in agentic data marketplace interactions without quantitative benchmarks comparable to DataGovBench.

13.3 Data and Tools

Many studies in LLM-based data governance and cataloging rely on proprietary enterprise datasets, which restricts reproducibility and direct comparative evaluations due to privacy, compliance, and sensitivity concerns (e.g., internal schemas, personally identifiable information, or policy details).

However, approaches using curated, grounded, or synthetic elements are gaining traction to enable standardized and reproducible testing of agents. The two focal works exemplify this balance between realism and methodological rigor.

In **DataGovBench** [57], the benchmark addresses reproducibility through a carefully designed, semi-automated pipeline that avoids direct exposure of proprietary data. It utilizes 30 tables sourced from Statista (2025), covering diverse domains (e.g., tourism, e-commerce, sports), with only task-relevant columns retained along with necessary confounding columns to preserve semantic integrity. To introduce realistic governance challenges, a novel *reversed-objective* methodology inverts original task goals (e.g., from imputation to deliberate missing-value insertion) and programmatically generates task-specific noise (e.g., duplicates, inconsistencies, mislabels) on these base tables. Manual validation prevents hallucinated or excessive noise.

The 150 tasks are constructed hierarchically: 100 single-operator (atomic) tasks annotated by domain experts, followed by the worst-performing ones combined into 50 coherent multi-step Directed Acyclic Graphs (DAGs) with newly formulated natural-language objectives and additional noise injection. Evaluation is fully automated via task-specific Python scripts (validated against ground truth) that compute standardized metrics: **Code Runnable Rate (CRR)**, **Task Success Rate (TSR)**, and **Average Task Score (ATS)**. While the full dataset and code are not publicly released in the paper (no GitHub or Hugging Face links are provided), the reversed-objective synthesis and script-based automation support transparency and potential replication of the evaluation framework.

The associated **DataGovAgent** serves as a state-of-the-art orchestration tool, integrating LLM reasoning with symbolic elements (constraint-guided planning via contracts and pre/post-conditions), retrieval-augmented generation (RAG over a curated library such as DCAI), and sandboxed execution for safe, iterative debugging and feedback loops. This modular design enables embedding

within broader governance workflows, including interaction with data catalogs, profiling tools, and automated pipelines.

In contrast, **Data Product MCP** [98] operates exclusively on real enterprise data products within a governed marketplace (e.g., via Entropy Data / Data Mesh Manager), with no synthetic datasets or public benchmarks. Data remains hosted in connected sources (Snowflake, Databricks, GCP, Azure, AWS), while metadata, data contracts (following the Open Data Contract Standard – ODCS), and governance rules are synchronized to the marketplace.

The system provides an open-source **Data Product MCP Server** (Python-based, built on FastMCP; source code available at <https://github.com/entropy-data/dataproduct-mcp>), which exposes modular, agent-accessible tools:

- **Data Product Search** – semantic and list-based discovery by ID, title, description, or business context;
- **Data Product Get** – retrieval of full metadata, connections, and contracts;
- **Data Product Request Access** – submission of purpose-declared access requests (with manual or automatic approval workflows);
- **Data Product Query** – real-time validation by Governance AI (powered by GPT-4o) against contracts, global policies, and purpose limitations before executing SQL on source systems.

This extensible plugin and API architecture embeds LLM agents as production-ready components for continuous metadata management, access control, quality monitoring (via contracts), and proactive policy validation (violation blocking and immutable audit trails). Evaluation is qualitative, based on expert surveys ($n = 16$) and scenario demonstrations, emphasizing usability, governance preservation, and workflow impact rather than quantitative benchmarks or synthetic data.

Overall, these works illustrate complementary directions: DataGovBench advances reproducible, synthetic-noise-driven benchmarking for governance workflows, while Data Product MCP prioritizes practical, open-source tool integration with live enterprise catalogs – both reducing full dependence on proprietary setups and enabling agent embedding in governed data environments.

13.4 Future direction

Future research should focus on:

- **Trust and Verification:** Combining LLMs with formal verification or human-in-the-loop systems to ensure safe and reliable decisions.
- **Explainability:** Improving interpretability and traceability of LLM actions, especially for auditing and compliance.
- **Cost-Efficiency:** Optimizing multi-agent orchestration and hybrid model use to reduce computational costs.
- **Cross-Domain Generalization:** Building domain-adaptive LLM agents that retain performance across varied governance scenarios.

The synergy of LLMs with traditional data management frameworks promises to revolutionize data governance, but careful system design and ongoing evaluation remain crucial.

14 Semantic Web & Knowledge Engineering using LLM agents

Semantic Web & Knowledge Engineering using LLM agents Semantic Web is about making data machine readable. It is presenting a string of text as entities and relationships with well defined meaning. On the other hand, Knowledge Engineering is the discipline of designing, constructing, and maintaining structured knowledge representations that systems can reason over. Knowledge engineering involves ontologies, schemas, and constraints. The current challenges in this field

revolves around scalability, dynamic ontology evolution, reasoning complexity, and data interoperability within large and heterogeneous datasets [94]. AI Agents embedded with LLM can help automate ontology learning, enhance reasoning over complex semantic structures, and reduce manual knowledge engineering effort.

14.1 Survey

The advent of Agentic AI enables developers to hand off their jobs to AIs with simple prompt instructions. Agentic AIs are equipped with reasoning, tool invocation, memory, and iteration, and can therefore serve as assistant tools for developers to build projects.

The existing research on employing LLM agents for the Semantic Web and Knowledge Engineering proposes a wide range of agent design space in terms of the architecture, coordination, memory, and tool use. The LLM agents can be categorized in terms of the reasoning capabilities of the agents as well as the working mechanism. Single-agent frameworks, like **KG-Agent** [33] utilize the LLM agent in a loop of reasoning and execution for multi-step knowledge graph tasks by keeping the state of the intermediate reasoning process, and employing symbolic tools for validation. The author proposes KG as an autonomous agent framework for solving complex reasoning tasks over knowledge graphs. Another single agent framework is GR-Agent [131] which works on an interactive environment and has adaptive reasoning under incomplete knowledge graph which allows the agent to proceed even when they are missing out on some facts.

In multi-agent architectures, different agents communicate with each other and specialize in different subtasks. In a fully-decentralized multi-agent architecture, **AgentNet** [118] studies a scenario in which multiple agents jointly reason about a complex question. Each agent specializes in different parts of the task according to the task context and shares abstract representations with other agents through a communication mechanism. No manager agent is required, making the architecture scalable to multiple knowledge bases.

In role-based multi-agent systems, **ChattyKG** [71] assigns distinct roles (contextual understanding, entity and relation extraction, structured query planning, and execution) to handle multi-turn dialogue over knowledge graphs. **KARMA** [58] proposes a cooperative multi-agent framework for knowledge graph enrichment, where specialized agents handle entity disambiguation, relation suggestion, knowledge validation, and conflict resolution through iterative interactions.

A recent multi-agent collaborative framework for constructing knowledge graphs directly from text is presented by Chen and Liu [14]. Their system employs multiple collaborating agents that work together to extract entities and relations from unstructured text, perform consistency checks, and incrementally populate the target knowledge graph, demonstrating strong performance on real-world corpora.

AGENTiGraph [127] further advances interactive knowledge graph construction by assigning roles such as dialogue understanding, graph editing, constraint validation, and graph revision, enabling collaborative construction and management through user interactions.

Although the above results demonstrate a good progress, these works also point out some limitations of LLM agents. They may violate the semantic properties or inject new relation for the new concepts, or uncontrollably evolve the ontology if they are not thoroughly verified. Therefore, in most of the proposed frameworks, the agents are heavily restricted by symbolic pre- and post-conditions, fixed tools and their execution monitoring results. In general, the above studies present a valuable potential of LLM-based agents in developing adaptive and scalable SW systems, yet, at the same time, they show the importance of better results assurance, evaluation metrics and long-term semantic coherence.

Table 12. LLM Agents for semantic web & Knowledge engineering

Method	Agent Type	Orchestration	Planning	Memory	Prompting	External Tools	Query / Action Strategy	Data Types
KG-Agent (G-Agent) [33]	(G- Single agent	Single-agent loop	Iterative reason-act planning	Intermediate reasoning traces	Chain-of-thought prompts	KG traversal and reasoning operators	Stepwise KG traversal and inference	Knowledge graphs
GR-Agent [131]	Single agent	Closed-loop execution	Adaptive multi-hop planning	Execution context	Reasoning-guided prompts	Graph interaction operators	Fallback and alternative path exploration	Incomplete knowledge graphs
From Text to Trust [64]	Multi-agent	Agent-inspired pipeline	Task decomposition + verification loop	Intermediate extraction state	Domain-specific prompts	LLM extraction + embedding similarity tools	Embedding-based semantic verification	Unstructured text → ADAS KGs
AgentNet [127]	Multi-agent	Decentralized coordination	Dynamic agent specialization	Distributed agent states	Role-agnostic prompts	Inter-agent communication interfaces	Collaborative reasoning actions	Distributed semantic sources
ChattyKG [71]	Multi-agent	Role-based coordination	Pipeline-level planning	Dialogue context	Role-specific prompts	SPARQL execution interfaces	Semantic parsing to structured queries	Knowledge graphs
KARMA [58]	Multi-agent	Pipeline-style coordination	Explicit task decomposition	Shared enrichment state	Role-conditioned prompts	KG update and validation operators	Entity and relation enrichment actions	Knowledge graphs + text
AGENTiGraph [127]	Multi-agent	Interactive coordination	User-driven iterative planning	Evolving graph state	Dialogue-driven prompts	Graph modification operations	Intent-to-graph action translation	Knowledge graphs
Chen & Liu [14]	Multi-agent	Collaborative pipeline	Joint entity-relation extraction	Shared construction state	Collaborative prompts	Extraction + consistency check operators	Incremental KG population from text	Unstructured text → Knowledge graphs

14.2 Metrics and Benchmarks

The assessment of agentic systems in the Semantic Web and Knowledge Engineering area is multifaceted given the heterogeneity of tasks such as reasoning, question answering, enrichment, and collaborative construction. Consequently, individual contributions make use of task-specific accuracy metrics, reasoning quality measures, and interaction level metrics but no unified benchmark is present.

In the case of knowledge graph reasoning and question answering agents (G-Agent, GR-Agent and ChattyKG), answer correctness is the main metric of evaluation. For the former, this involves calculating exact match, F1 score and hits@k of the ranked inference results, usually on multi-hop reasoning datasets that are built on top of a known knowledge graph, where the performance of the agent hinges on its capability to perform relational reasoning and to integrate intermediate results, especially in a scenario where the knowledge graph is incomplete.

Performance metrics for cooperative multi-agent reasoning frameworks like AgentNet extend beyond solution correctness to include metrics of coordination, such as task completion percentage, convergence time, and perhaps worst-case growth with respect to the number of agents or difficulty of the task. The systems are typically compared on synthetic or composite problems crafted to test the limits of decentralized problem-solving and specialization.

Finally, in the case of systems focused on enriching knowledge graphs, such as KARMA and Generate-on-Graph, the main aspect of interest is the correctness of the generated or validated knowledge. As such, typical evaluation metrics include the precision, recall, and F1 score of the entities and relations extracted by a system from a text document with respect to some reference knowledge graph, or the number of consistent or conflicting modifications induced by the new knowledge on the original knowledge graph. The evaluation of such systems is typically done over selected, or carefully-designed examples from text-to-KG benchmarks, or manually-validated enrichment tasks.

In the case of interactive and human-in-the-loop methods like AGENTiGraph, structural correctness of the graph updates is of course still important, but interaction success rate and semantic coherence between turns also matter, as does the system's capability to understand and correct its

understanding of the user's intent across the interaction. Often, scenario-based evaluation rather than a fixed leaderboard is used, to reflect the focus on usability and graph modification.

In summary, the absence of common metrics for all tasks poses one of the problems here. These metrics are useful for evaluating the task-specific performance, but they do not evaluate important aspects of agentic systems such as the degree of coordination among the agents, robustness to incomplete knowledge, or semantic coherence over long horizons. This is why there is still a push towards more general frameworks for evaluating agentic knowledge.

14.3 Data and Tools

Unlike solely learned representations from the training data, agentic LLM based approaches for SW and KE leverage a combination of pre-trained models, structured and semantic data, and tool-based execution in order to provide reasoning, enrichment, and tool-supported knowledge curation capabilities.

In most systems, KGs and ontologies, which organize entities, relationships, and their semantic relationships in a structured format, are employed as the key knowledge source. For the single-agent cases, including G-Agent, GR-Agent, Generate-on-Graph, and ChattyKG, the pre-defined KG is assumed as external semantic memory for a multi-hop reasoning or inference. They may be (partially) empty or sparsely filled. In such a case, an agent must reason about a partial relational structure, rather than a fully defined schema.

There are also multi-agent systems such as AgentNet that can potentially be designed to work on multiple or decentralized knowledge graphs and/or semantic schemas (depending on the task) where each agent has access to a different portion of the structured data or a different planning problem. Similarly, a knowledge graph completion algorithm such as KARMA or Generate-on-Graph will also have to consider multiple knowledge sources in addition to the graphs that they work with, including semi-structured or natural language sources (e.g., text) that contain the new information that is being integrated into the graph. Another example could be the AGENTiGraph algorithm that, in addition to the knowledge graph, also considers natural language sources in the form of the text generated through user interactions.

The LLMs underlying all systems are usually pre-trained on massive text data, and not fine-tuned on the knowledge graph data. The structured knowledge is injected at inference time by interacting with these external knowledge bases.

Furthermore, in order to operate on these data sources, an agent must be equipped with an explicit execution mechanism that provides her with the ability to, for instance, query, navigate, manipulate, or add to a structured knowledge graph. In a single-agent scenario, this mechanism facilitates repeated cycles of reasoning, acting, and observing: The agent can perform a graph operation, observe the result, and decide on her next action on the basis of what she has observed.

In contrast, in other decentralized multi-agent systems (e.g., AgentNet [Katifori et al. 2018]), tools operate as solitary activities per agent, and the agents coordinate by sharing information without controlling one another. In other role-based multi-agent systems (e.g., ChattyKG [Balog et al. 2018] and KARMA [Vernetti and Gray 2016]), tool behaviors pertain to the agent's semantic role, enabling different agents to perform different activities for the same task, e.g., interpretation, mapping, validation, or integration, while keeping their common model consistent.

For tools on interactive platforms like AGENTiGraph, they help agents ground human input into specific actions on a graph and gradually incorporate user input into a knowledge graph. For all, these execution tools facilitate the grounding of LLM reasoning on a formal semantic representation to provide transparent, interpretable, and controlled methods of iterative knowledge engineering

14.4 Future Direction

Despite the potential of the state-of-the-art agentic systems for reasoning, enrichment, and interactive knowledge management, most of these systems are task-dependent and disconnected. There is a need for agent architectures which can adapt their execution strategies based on the task types (reasoning, construction, maintenance) and the knowledge graph evolution. Similarly, in multi-agent systems, agents should be able to coordinate with each other and adapt their roles and responsibilities when the knowledge is incomplete, schema is changing, or when data is from different sources. Moreover, there is a lack of common metrics for evaluating the long-term reasoning performance, coordination between agents, and semantic coherence of the knowledge graph. Such metrics should be defined to mimic real-world knowledge engineering scenarios. Another important direction is to integrate symbolic and connectionist AI to build agents that can reason about knowledge graphs and also reason about the knowledge graph structure, its uncertainty, and evolution.

15 Conclusion

This paper introduces the increasing adaption of AI in data management. It offers a comprehensive framework comprising 11 subareas. This paper presents how single-agent and multi-agent systems differ. It shows that in every subarea multi-agent outperforms single-agent systems. According to a study done by Anthropic [6] 90% of complex tasks involving multi-agent outperform single-agent, but they are more computationally intensive, consume more tokens, and require non-trivial coordination among the multiple agents, which the LLMs struggle with. Single-agent systems are easier to configure but cannot decompose complex tasks into simpler ones. The paper presents challenges and opportunities from all subareas - from integration with rich data tools, to ensuring efficient and scalable pipelines - that tie these tasks together as a cohesive research agenda.

While progress is being made quickly, there is still a long way to go before we can rely on agentic data tools in production environments. For example, we still lack standard evaluation metrics and benchmarks: in a recent paper, the authors point out that “the currently fragmented evaluation landscape for agentic AI should be consolidated.” In reality, “there is currently little agreement on how to evaluate agentic behaviour,” and no established benchmarks for multistep data workflows, “which makes comparing different approaches challenging.” On top of this, LLM-enabled agents are known to hallucinate (i.e., fabricate) plausible-sounding but factually incorrect data, which is a natural consequence of their probabilistic language model nature. Hallucinated data can spread throughout a data pipeline unless it’s actively detected and removed. Coordinating the behavior of multiple data agents is another challenge: teams of agents present new challenges for synchronization, reliability, and tool use. According to Anthropic engineers, “it’s essential to think deliberately about how LLM agents can be composed, as they are not yet great at coordinating and delegating to other agents in real time.” To sum this up, today’s agentic data tools typically require significant human intervention, custom tooling, and bespoke evaluation protocols to ensure correctness — another indicator of how early we are in the maturity and standardization of the field.

Meanwhile, agentic data management is also gaining traction in industry. AI agents are already being used to automate common data tasks and support analytics work. For example, Informatica describes agentic data platforms that “automate repetitive and time-consuming tasks, freeing up valuable time for strategic initiatives”. They list examples of agents that can “scan for data quality issues”, “trigger ETL jobs”, and “apply data governance policies in real-time”. This promises to deliver significant productivity improvements and more agile data management. Informatica also mentions that filling a skills vacuum is another major benefit: agents can “democratize data management” by

allowing non-technical users to “query and manipulate data using natural language”. Of course, this also introduces new risks: agent decision-making can be difficult to interpret, which raises problems of explainability and trust. In any case, AI agents are clearly playing an increasingly important role in actual data pipelines, changing the way that data engineers and analysts operate.

There are a few areas of future work that are important to make agentic data systems a reality. The most important is the development of standardized benchmarks that measure not just efficacy (or accuracy) of agentic data systems but also their efficiency, robustness and cost. The recently proposed CLEAR framework proposes that we should benchmark our agents on the dimensions of Cost, Latency, Efficacy, Assurance and Reliability. We will also need new benchmarking datasets that model real data processing pipelines and safety requirements to test our agents.

Second, hybrid models that combine neural LLMs with symbolic control may offer better robustness: e.g., the SCL model adds a symbolic “governance layer” to the LLM planning process. This imposes rule-based constraints on the planner. The model separates the reasoning into the retrieval, cognition, control and memory components and is thus able to achieve 100 traceability of decisions and 0 logical violations on a given benchmark. In general, incorporating domain knowledge and logic into LLMs could aid an agent in making verifiably correct decisions and explaining them.

Third, closer integration with data tools and systems will be needed. Next-generation agents should be able to connect directly to databases, data lakes, and ML pipelines and call APIs or issue queries as needed. For example, in the Informatica stack, an agent can convert a user’s intent (“show me last quarter’s sales report”) into specific tasks (access sales database; filter by date; compute sums; draw chart) and execute them. Similar functionality will need hundreds of connectors (to databases; BI tools; cloud services) and automatic data extraction and preparation steps. Advances in retrieval-augmented generation and RAG pipelines — enriching LLM input with relevant information — will also help with accuracy and reducing hallucination risk.

Fourth, agents must be made trustworthy and explainable. For example, they must be able to source what they say or provide information on the provenance of the data, express when they are unsure, and allow users to give feedback on their responses. Simple things such as adding a retriever or a knowledge base to agents that are more capable of providing such explanations or encouraging agents to say that they don’t know something can also help in avoiding hallucination. Other methods such as constrained decoding, validating agent outputs, and making agents evaluate their own outputs can help in minimizing hallucination. Methods for providing formal guarantees on certain agentic behavior in certain sensitive pipelines or monitoring the pipeline for agentic behavior should be explored.

Third, we expect that mixed-initiative collaboration between humans and AI systems will continue to play an essential role in data-intensive settings. In particular, humans and agents can collaborate to accomplish tasks, with each playing to their respective strengths. Humans, for example, may provide high-level objectives, define domain constraints, and validate outputs, while agents perform data- and reasoning-intensive tasks. In fact, some of the recent efforts on human–AI collaborative workflows have proposed frameworks for structured, interactive collaboration between humans and AI systems, in which control and provenance information can be maintained throughout the process. Within such frameworks, a mixed-initiative approach, where agents propose (e.g., learned) actions and humans are able to modify or otherwise control these proposals, offers the potential to balance responsiveness with the ability to exert control.

In conclusion, this survey has presented a wide variety of agentic data management techniques and identified their strengths and weaknesses. There is potential for major improvements, as today, LLMs enable logical reasoning, however, their successful integration into data flows will necessitate continued engineering efforts, testing, and interdisciplinary research. We envision that our paper will provide the stimulus for future work on benchmarking, symbolic reasoning, trustworthiness,

and human-AI teams. Ultimately, this will allow agentic systems to mature to industrial-grade levels and to enable future intelligent, automated data management.

References

- [1] Ciara Adkins. Llmatch algorithm 2.0: A methodology for ranking and recommending language models. <https://github.com/ciaraadkins/llmatch>, 2024.
- [2] Aatif Muhammad Althaf, Muzakkiruddin Ahmed Mohammed, Mariofanna Milanova, John Talburt, and Mert Can Cakmak. Multi-agent rag framework for entity resolution: Advancing beyond single-llm approaches with specialized agent coordination. *Computers (2073-431X)*, 14(12), 2025.
- [3] Rami Aly, Zhijiang Guo, Michael Sejr Schlichtkrull, James Thorne, Andreas Vlachos, Christos Christodoulopoulos, Oana Cocarascu, and Arpit Mittal. The fact extraction and verification over unstructured and structured information (feverous) shared task. In *Proceedings of the Fourth Workshop on Fact Extraction and VERification (FEVER)*. Association for Computational Linguistics, 2021.
- [4] Michael Armbrust, Ali Ghodsi, Reynold Xin, Matei Zaharia, et al. Lakehouse: a new generation of open platforms that unify data warehousing and advanced analytics. In *Proceedings of CIDR*, volume 8, page 28. sn, 2021.
- [5] Suresh Babu Avula. Oracle autonomous database for transforming database administration by harnessing artificial intelligence. In *2025 Global Conference in Emerging Technology (GINOTECH)*, pages 1–7. IEEE, 2025.
- [6] Kushal Banda. How we built our multi-agent research system — anthropic. *Medium*, 2025. <https://ai.plainenglish.io/how-we-built-our-multi-agent-research-system-5f5e10b2a8d6>.
- [7] BIRD-bench Team. Bird-bench: A big benchmark for large-scale database grounded text-to-sql. <https://bird-bench.github.io/>, 2026. Accessed: 2026-01-20.
- [8] Jan-Micha Bodensohn, Ulf Brackmann, Liane Vogel, Anupam Sanghi, and Carsten Binnig. Unveiling challenges for LLMs in enterprise data engineering. *Proc. VLDB Endow.*, 19(2):196–209, 2025.
- [9] Ben Bogin, Jonathan Berant, and Matt Gardner. Representing schema structure with graph neural networks for text-to-sql parsing. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4560–4565, 2019.
- [10] Rohan Bopardikar, Jin Wang, and Jia Zou. Structured multi-step reasoning for entity matching using large language model. *arXiv preprint arXiv:2511.22832*, 2025.
- [11] Jipeng Cen, Jiaxin Liu, Zhixu Li, and Jingjing Wang. Sqlfixagent: Towards semantic-accurate text-to-sql parsing via consistency-enhanced multi-agent collaboration. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 49–57, 2025.
- [12] Shuaichen Chang, Jun Wang, Marvin Dong, Lin Pan, Henghui Zhu, Alexander Hanbo Li, Wuwei Lan, Sheng Zhang, Jiarong Jiang, Joe Lilien, et al. Dr. spider: A diagnostic evaluation benchmark towards text-to-sql robustness. *Amazon Science*, 2023.
- [13] Shuaichen Chang, Jun Wang, Mingwen Dong, Lin Pan, Henghui Zhu, Alexander Hanbo Li, Wuwei Lan, Sheng Zhang, Jiarong Jiang, Joseph Lilien, et al. Dr. spider: A diagnostic evaluation benchmark towards text-to-sql robustness. In *The Eleventh International Conference on Learning Representations*, 2023.
- [14] Gui Chen and Xianhui Liu. A multi-agent collaborative framework for constructing knowledge graphs from text. In *2024 IEEE International Conference on Knowledge Graph (ICKG)*, pages 9–16, 2024.
- [15] Ke Chen, Peiran Wang, Yaoning Yu, Xianyang Zhan, and Haohan Wang. Large language model-based data science agent: A survey. *arXiv preprint arXiv:2508.02744*, 2025.
- [16] Zhutian Chen, Chenyang Zhang, Qianwen Wang, Jakob Troidl, Simon Warchol, Johanna Beyer, Nils Gehlenborg, and Hanspeter Pfister. Beyond generating code: Evaluating gpt on a data visualization course. In *2023 IEEE VIS Workshop on Visualization Education, Literacy, and Activities (EduVis)*, pages 16–21. IEEE, 2023.
- [17] Liying Cheng, Xingxuan Li, and Lidong Bing. Is gpt-4 a good data analyst? *arXiv preprint arXiv:2305.15038*, 2023.
- [18] Xu Chu, John Morcos, Ihab F Ilyas, Mourad Ouazzani, Paolo Papotti, Nan Tang, and Yin Ye. Katara: A data cleaning system powered by knowledge bases and crowdsourcing. In *Proceedings of the 2015 ACM SIGMOD international conference on management of data*, pages 1247–1261, 2015.
- [19] Brian F Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with ycsb. In *Proceedings of the 1st ACM symposium on Cloud computing*, pages 143–154, 2010.
- [20] Valter Crescenzi, Andrea De Angelis, Donatella Firmani, Maurizio Mazzei, Paolo Meriardo, Federico Piai, and Divesh Srivastava. Alaska: A flexible benchmark for data integration tasks. *CoRR*, 2021.
- [21] Zikun Cui, Tianyi Huang, Chia-En Chiang, and Cuiqianhe Du. Toward verifiable misinformation detection: A multi-tool llm agent framework. In *Proceedings of the 2025 International Conference on Generative Artificial Intelligence for Business*, pages 179–185, 2025.

- [22] Djellel Eddine Difallah, Andrew Pavlo, Carlo Curino, and Philippe Cudre-Mauroux. Oltp-bench: An extensible testbed for benchmarking relational databases. *Proceedings of the VLDB Endowment*, 7(4):277–288, 2013.
- [23] R Ding, S Han, and D Zhang. Insightpilot: An llm-empowered automated data exploration system. In *EMNLP 2023. ACL special interest group on linguistic data (SIGDAT)*, 2023.
- [24] Saeed Fathollahzadeh, Essam Mansour, and Matthias Boehm. Catdb: Data-catalog-guided, llm-based generation of data-centric ml pipelines. *Proceedings of the VLDB Endowment*, 18(8):2639–2652, 2025.
- [25] Alvaro A. A. Fernandes, Martin Koehler, Nikolaos Konstantinou, Pavel Pankin, Norman W. Paton, and Rizos Sakellariou. Data preparation: A technological perspective and review. *SN Comput. Sci.*, (4):425, 2023.
- [26] Aditya Gautam. Multi-agent systems for misinformation lifecycle: Detection, correction and source identification. *arXiv preprint arXiv:2505.17511*, 2025.
- [27] Victor Giannakouris and Immanuel Trummer. λ -tune: Harnessing large language models for automated database system tuning. *Proc. ACM Manag. Data*, 3(1), February 2025.
- [28] Yale Semantic Parsing Group. Spider: A large-scale human-labeled text-to-sql dataset. <https://yale-lily.github.io/spider>, 2018. Accessed: 2026-01-20.
- [29] Siyuan Guo, Cheng Deng, Ying Wen, Hechang Chen, Yi Chang, and Jun Wang. Ds-agent: automated data science by empowering large language models with case-based reasoning. In *Proceedings of the 41st International Conference on Machine Learning, ICML'24*. JMLR.org, 2024.
- [30] Sirui Hong, Yizhang Lin, Bang Liu, Bangbang Liu, Binhao Wu, Ceyao Zhang, Danyang Li, Jiaqi Chen, Jiayi Zhang, Jinlin Wang, et al. Data interpreter: An llm agent for data science. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 19796–19821, 2025.
- [31] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv e-prints*, pages arXiv–2410, 2024.
- [32] IEEE-CIS and Kaggle. Ieee-cis fraud detection. <https://www.kaggle.com/c/ieee-fraud-detection>, 2019.
- [33] Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, Yang Song, Chen Zhu, Hengshu Zhu, and Ji-Rong Wen. Kg-agent: An efficient autonomous agent framework for complex reasoning over knowledge graph. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9505–9523, 2025.
- [34] Tengjun Jin, Yuxuan Zhu, and Daniel Kang. Elt-bench: An end-to-end benchmark for evaluating ai agents on elt pipelines. *arXiv preprint arXiv:2504.04808*, 2025.
- [35] M Joshi and VK Madiseti. Intelligent etl for enterprise software applications using unstructured data [j]. *Journal of Software Engineering and Applications*, 18(1):4465, 2025.
- [36] Kaggle. Titanic: Machine learning from disaster. <https://www.kaggle.com/c/titanic>, 2012.
- [37] Kaggle. House prices: Advanced regression techniques. <https://www.kaggle.com/c/house-prices-advanced-regression-techniques>, 2016.
- [38] Arijit Khan, Yuyu Luo, Wenjie Zhang, Minqi Zhou, and Xiaofang Zhou. Retrieval-augmented generation (rag): What is there for data management researchers? a discussion on research from a panel at llm+vector data workshop @ ieee icde 2025. *SIGMOD Rec.*, 54(4):33–42, 2026.
- [39] Alexey Kopytov. Sysbench: A scriptable multi-threaded benchmark tool. <https://github.com/akopytov/sysbench>, 2023.
- [40] Sanjay Krishnan, Daniel Haas, Michael J. Franklin, and Eugene Wu. Towards reliable interactive data cleaning: a user survey and recommendations. In *Proceedings of the Workshop on Human-In-the-Loop Data Analytics (HILDA@SIGMOD)*, 2016.
- [41] Kumud Lakara, Georgia Channing, Juil Sock, Christian Rupprecht, Philip Torr, John Collomosse, and Christian Schroeder de Witt. Llm-consensus: Multi-agent debate for visual misinformation detection. *arXiv preprint arXiv:2410.20140*, 2024.
- [42] Kumud Lakara, Juil Sock, Christian Rupprecht, Philip Torr, John Collomosse, and Christian Schroeder de Witt. Mad-sherlock: Multi-agent debates for out-of-context misinformation detection. 2024.
- [43] Jiale Lao and Immanuel Trummer. GenDB: The next generation of query processing – synthesized, not engineered, 2026. URL <https://arxiv.org/abs/2603.02081>. arXiv preprint (submitted 2 March 2026).
- [44] Viktor Leis, Bernhard Radke, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. Query optimization through the looking glass, and what we found running the join order benchmark. *The VLDB Journal*, 27(5):643–668, 2018.
- [45] Hui Li, Ante Wang, Kunquan Li, Zhihao Wang, Liang Zhang, Delai Qiu, Qingsong Liu, and Jinsong Su. A multi-agent framework with automated decision rule optimization for cross-domain misinformation detection. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, November 2025.
- [46] Lan Li, Liri Fang, Bertram Ludäscher, and Vetle I Torvik. AutoDCWorkflow: LLM-based data cleaning workflow auto-generation and benchmark. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet

- Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 7766–7780. Association for Computational Linguistics, 2025.
- [47] Xinyi Li, Yongfeng Zhang, and Edward C Malthouse. Large language model agent for fake news detection. *arXiv preprint arXiv:2405.01593*, 2024.
 - [48] Yiyan Li, Haoyang Li, Jing Zhang, Renata Borovica-Gajic, Shuai Wang, Tieying Zhang, Jianjun Chen, Rui Shi, Cuiping Li, and Hong Chen. Agenttune: An agent-based large language model framework for database knob tuning. *Proc. ACM Manag. Data*, 3(6), 2025.
 - [49] Zhaodonghui Li, Haitao Yuan, Jiachen Shi, Hao Zhang, Yu Rong, and Gao Cong. Amaze: A multi-agent zero-shot index advisor for relational databases. *arXiv preprint arXiv:2508.16044*, 2025.
 - [50] Ziming Li, Qianbo Zang, David Ma, Jiawei Guo, Tianyu Zheng, minghao liu, Xinyao Niu, Xiang Yue, Yue Wang, Jian Yang, Jiaheng Liu, Wanjuan Zhong, Wangchunshu Zhou, Wenhao Huang, and Ge Zhang. Autokaggle: A multi-agent framework for autonomous data science competitions, 2025. URL <https://openreview.net/forum?id=09LEjbLcZW>.
 - [51] Hongzhan Lin, Yang Deng, Yuxuan Gu, Wenxuan Zhang, Jing Ma, See Kiong Ng, and Tat-Seng Chua. Fact-audit: An adaptive multi-agent framework for dynamic fact-checking evaluation of large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 360–381, 2025.
 - [52] Qi Lin, Zhenyu Zhang, Viraj Thakkar, Zhenjie Sun, Mai Zheng, and Zhichao Cao. Storagextuner: An llm agent-driven automatic tuning framework for heterogeneous storage systems. *arXiv preprint arXiv:2510.25017*, 2025.
 - [53] Pei Liu, Xin Liu, Ruoyu Yao, Junming Yan, Siyuan Meng, Ding Wang, and Jun Ma. Hm-rag: Hierarchical multi-agent multimodal retrieval augmented generation. In *Proceedings of the 33rd ACM international conference on multimedia*, pages 2781–2790, 2025.
 - [54] Xuannan Liu, Zekun Li, Pei Pei Li, Huaibo Huang, Shuhan Xia, Xing Cui, Linzhi Huang, Weihong Deng, and Zhaofeng He. MMFakebench: A mixed-source multimodal misinformation detection benchmark for LVLs. In *The Thirteenth International Conference on Learning Representations*, 2025.
 - [55] Ye Liu, Yue Xue, Daoyuan Wu, Yuqiang Sun, Yi Li, Miaolei Shi, and Yang Liu. PropertyGPT: LLM-driven formal verification of smart contracts through retrieval-augmented property generation. In *Network and Distributed System Security Symposium (NDSS)*, 2025. doi: 10.14722/ndss.2025.241357.
 - [56] Yuhao Liu, Yuxuan Liu, Xiaoqing Zhang, Xiuying Chen, and Rui Yan. The truth becomes clearer through debate! multi-agent systems with large language models unmask fake news. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 504–514, 2025.
 - [57] Zhou Liu, Zhaoyang Han, Guochen Yan, Hao Liang, Bohan Zeng, Xing Chen, Yuanfeng Song, and Wentao Zhang. Datagovbench: Benchmarking llm agents for real-world data governance workflows. *arXiv preprint arXiv:2512.04416*, December 2025. URL <https://arxiv.org/abs/2512.04416>. Version 2 (v2).
 - [58] Yuxing Lu, Wei Wu, Xukai Zhao, Rui Peng, and Jinzhuo Wang. KARMA: Leveraging multi-agent LLMs for automated knowledge graph enrichment. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=k0wyi4cOGy>.
 - [59] Grace Luo, Trevor Darrell, and Anna Rohrbach. Newsclippings: Automatic generation of out-of-context multimodal media. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 6801–6817, 2021.
 - [60] Jiatong Ma, Linmei Hu, Rang Li, and Wenbo Fu. Local: Logical and causal fact-checking with llm-based multi-agents. In *Proceedings of the ACM on Web Conference 2025*, pages 1614–1625, 2025.
 - [61] Qingzhi Ma, Ali M Shanghooshabad, Mehrdad Almasi, Meghdad Kurmanji, and Peter Triantafillou. Learned approximate query processing: Make it light, accurate and fast. In *Conference on Innovative Data Systems (CIDR21)*, 2021.
 - [62] Xiaochen Ma, Guozheng Rao, Lina Xu, Xin Wang, Zaiming Fan, and Zhe Zhang. Guided and knowledgeable multi-agent debate for fact verification. *Expert Systems with Applications*, page 130103, 2025.
 - [63] Mohammad Mahdavi and Ziawash Abedjan. Semi-supervised data cleaning with raha and baran. In *CIDR*, 2021.
 - [64] María del Rocío Martínez Rodríguez. *From Text to Trust: An LLM Multi-Agent System with Embedding Verification for ADAS Knowledge Graph Construction*. PhD thesis, Jönköping University, Jönköping AI Lab (JAIL), June 2025. URL <http://hj.diva-portal.org/smash/record.jsf?pid=diva2:1972482>. Master’s thesis.
 - [65] Wes McKinney. Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference (SciPy)*, pages 51–56, 2010.
 - [66] Kai Nakamura, Sharon Levy, and William Yang Wang. Fakeddit: A new multimodal benchmark dataset for fine-grained fake news detection. In *Proceedings of the twelfth language resources and evaluation conference*, pages 6149–6157, 2020.
 - [67] Binye Ni, Xinlei Cai, Zhijun Shen, Zijie Meng, Junhua Zhao, Yuheng Cheng, and Xuanang Gui. Intelli-dispatch-sql: An llm-based agent for reliable text-to-sql in power dispatching. *Energy and AI*, page 100591, 2025.
 - [68] Farhad Nooralahzadeh, Yi Zhang, Jonathan Furst, and Kurt Stockinger. Explainable multi-modal data exploration in natural language via llm agent. *arXiv preprint arXiv:2412.18428*, 2024.

- [69] Mohammad Nour Al Awad, Sergey Ivanov, Olga Tikhonova, and Ivan Khodnenko. A multimodal conversational agent for tabular data analysis. *arXiv e-prints*, pages arXiv–2511, 2025.
- [70] Samuel Ojuri, The Anh Han, Raymond Chiong, and Alessandro Di Stefano. Optimizing text-to-sql conversion techniques through the integration of intelligent agents and large language models. *Information Processing & Management*, 62(5):104136, 2025.
- [71] Reham Omar, Abdelghny Orogat, Ibrahim Abdelaziz, Omij Mangukiya, Panos Kalnis, and Essam Mansour. Chatty-kg: A multi-agent ai system for on-demand conversational question answering over knowledge graphs. *arXiv preprint arXiv:2511.20940*, 2025.
- [72] VESPER OWEL. Natural language querying of databases: an information extraction approach in the conceptual query language. *International Journal of Human-Computer Studies*, 53(4):439–492, 2000.
- [73] George Papadakis, Nishadi Kirielle, Peter Christen, and Themis Palpanas. A Critical Re-evaluation of Record Linkage Benchmarks for Learning-Based Matching Algorithms. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pages 3435–3448, 2024.
- [74] Stefanos-Iordanis Papadopoulos, Christos Koutlis, Symeon Papadopoulos, and Panagiotis C Petrantonakis. Verite: a robust benchmark for multimodal misinformation detection accounting for unimodal bias. *International Journal of Multimedia Information Retrieval*, 13(1):4, 2024.
- [75] Md Rizwan Parvez, Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. Retrieval augmented code generation and summarization. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 2719–2734, 2021.
- [76] Andrew Pavlo, Gustavo Angulo, Joy Arulraj, Haibin Lin, Jiexi Lin, Lin Ma, Prashanth Menon, Todd C Mowry, Matthew Perron, Ian Quah, et al. Self-driving database management systems. In *CIDR*, volume 4, page 1, 2017.
- [77] Ralph Peeters, Reng Chiz Der, and Christian Bizer. WDC products: A multi-dimensional entity matching benchmark. In Letizia Tanca, Qiong Luo, Giuseppe Polese, Loredana Caruccio, Xavier Oriol, and Donatella Firmani, editors, *Proceedings 27th International Conference on Extending Database Technology*, pages 22–33, 2024.
- [78] Xuan-Quang Phan, Tan-Ha Mai, Thai-Duy Dinh, Minh-Thuan Nguyen, and Lam-Son Lê. Askdb: An llm agent for natural language interaction with relational databases. *arXiv preprint arXiv:2511.16131*, 2025.
- [79] Nele Pöldvere, Zia Uddin, and Aleena Thomas. The politifact-oslo corpus: A new dataset for fake news analysis and detection. *Information*, 14(12):627, 2023.
- [80] Danrui Qi, Zhengjie Miao, and Jiannan Wang. Cleanagent: Automating data standardization with llm-based agents. In *Proceedings of the VLDB Workshop on Data-Driven AI (DATAI)*, 2025. VLDB 2025.
- [81] Ruyi Qi, Zhou Liu, and Wentao Zhang. Datacross: A unified benchmark and agent framework for cross-modal heterogeneous data analysis. *arXiv preprint arXiv:2601.21403*, 2026.
- [82] Zhangcheng Qiang, Weiqing Wang, and Kerry Taylor. Agent-om: Leveraging llm agents for ontology matching. *Proceedings of the VLDB Endowment*, 18(3):516–529, 2024.
- [83] Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas, and Christopher Ré. Holoclean: Holistic data repairs with probabilistic inference. In *Proceedings of the 2017 International Conference on Management of Data (SIGMOD)*, pages 119–134. VLDB, 2017.
- [84] Salesforce Research. Wikisql: A large scale semantic parsing dataset for text-to-sql task. <https://github.com/salesforce/WikiSQL>, 2017. Accessed: 2026-01-20.
- [85] Ahmed Salem, Andrew Paverd, and Sahar Abdelnabi. Position: Stateless yet not forgetful: Implicit memory as a hidden channel in llms. In *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, 2026.
- [86] Aécio Santos, Eduardo HM Pena, Roque Lopez, and Juliana Freire. Interactive data harmonization with llm agents: Opportunities and challenges. In *NOVAS@SIGMOD/PODS*, 2025.
- [87] Mourad Sarrouti, Asma Ben Abacha, Yassine M’rabet, and Dina Demner-Fushman. Evidence-based fact-checking of health-related claims. In *Findings of the association for computational linguistics: EMNLP 2021*, pages 3499–3512, 2021.
- [88] Michael Schlichtkrull, Zhijiang Guo, and Andreas Vlachos. Averitec: A dataset for real-world claim verification with evidence from the web. *Advances in Neural Information Processing Systems*, 36:65128–65167, 2023.
- [89] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G. Parameswaran, and Eugene Wu. Docetl: Agentic query rewriting and evaluation for complex document processing. *Proc. VLDB Endow.*, 18(9):3035–3048, 2025.
- [90] Chen Shen, Jin Wang, Sajjadur Rahman, and Eser Kandogan. Demonstration of a multi-agent framework for text to sql applications with large language models. page 5280–5283. Association for Computing Machinery, 2024.
- [91] Significant Gravitas. Auto-GPT: An autonomous gpt-4 experiment. <https://github.com/Significant-Gravitas/AutoGPT>, 2023. Accessed: 2025-01-XX.
- [92] Vikramank Singh, Kapil Eknath Vaidya, Vinayshekhar Bannihatti Kumar, Sopan Khosla, Murali Narayanaswamy, Rashmi Gangadharaiah, and Tim Kraska. Panda: Performance debugging for databases using llm agents. 2024. URL <https://www.amazon.science/publications/panda-performance-debugging-for-databases-using-llm-agents>.

- [93] Yuyang Song, Hanxu Yan, Jiale Lao, Yibo Wang, Yufei Li, Yuanchun Zhou, Jianguo Wang, and Mingjie Tang. Quite: A query rewrite system beyond rules with llm agents. *arXiv preprint arXiv:2506.07675*, 2025.
- [94] Georgiana Stănescu (Nicolae) and Simona-Vasilica Oprea. Recent trends and insights in semantic web and ontology-driven knowledge representation across disciplines using topic modeling. *Electronics*, 2025.
- [95] Maojun Sun, Ruijian Han, Binyan Jiang, Houduo Qi, Defeng Sun, Yancheng Yuan, and Jian Huang. Lambda: A large model based data agent. *Journal of the American Statistical Association*, pages 1–13, 2025.
- [96] Zirui Tang, Weizheng Wang, Zihang Zhou, Yang Jiao, Bangrui Xu, Boyu Niu, Dayou Zhou, Xuanhe Zhou, Guoliang Li, Yeye He, et al. Llm/agent-as-data-analyst: A survey. *arXiv preprint arXiv:2509.23988*, 2025.
- [97] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. Fever: a large-scale dataset for fact extraction and verification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. Association for Computational Linguistics, 2018.
- [98] Marco Tonnamorelli, Filippo Scaramuzza, Simon Harrer, and Linus W. Dietz. Data product mcp: Chat with your enterprise data. *arXiv preprint arXiv:2601.08687*, January 2026. URL <https://arxiv.org/abs/2601.08687>. Version 1 (v1).
- [99] Transaction Processing Performance Council. *TPC BENCHMARK™ C: Standard Specification, Revision 5.11*. Transaction Processing Performance Council (TPC), 2010. Online; accessed 2025-11-29. URL: https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-c_v5.11.0.pdf.
- [100] Transaction Processing Performance Council. TPC benchmark H (tpc-h): Standard specification revision 2.17.1. Technical report, TPC, 2014. URL https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-h_v2.17.1.pdf.
- [101] Transaction Processing Performance Council. TPC benchmark DS (tpc-ds): Standard specification version 3.1.0. Technical report, TPC, 2021. URL https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-ds_v3.1.0.pdf.
- [102] Patara Trirat, Wonyong Jeong, and Sung Ju Hwang. Automl-agent: A multi-agent llm framework for full-pipeline automl. In *International Conference on Machine Learning*, pages 60099–60146. PMLR, 2025.
- [103] Christopher Troy, Sean Sturley, Jose M. Alcaraz-Calero, and Qi Wang. Enabling generative ai to produce sql statements: A framework for the auto- generation of knowledge based on ebnf context-free grammars. *IEEE Access*, 11:123543–123564, 2023.
- [104] Stefan Van Der Walt, S Chris Colbert, and Gael Varoquaux. The numpy array: a structure for efficient numerical computation. *Computing in science & engineering*, 13(2):22–30, 2011.
- [105] Joaquin Vanschoren, Jan N Van Rijn, Bernd Bischl, and Luis Torgo. Openml: networked science in machine learning. *ACM SIGKDD Explorations Newsletter*, 15(2):49–60, 2014.
- [106] Ashwin Verma, Alireza Sharbafchi, Behrouz Touri, and Soheil Mohajer. Distributed fact checking. In *2023 IEEE International Symposium on Information Theory (ISIT)*, pages 2649–2654. IEEE, 2023.
- [107] Ashwin Verma, Soheil Mohajer, and Behrouz Touri. Multi-agent fact-checker: Adaptive estimators. In *2024 IEEE 63rd Conference on Decision and Control (CDC)*, pages 6044–6050. IEEE, 2024.
- [108] N. Verma et al. Multi-agent fact checking. *arXiv*, 2025. URL <https://arxiv.org/abs/2503.02116>.
- [109] David Wadden, Kyle Lo, Bailey Kuehl, Arman Cohan, Iz Beltagy, Lucy Lu Wang, and Hannaneh Hajishirzi. Scifact-open: Towards open-domain scientific claim verification. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 4719–4734, 2022.
- [110] Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Bai, Linzheng Chai, Zhao Yan, Qian-Wen Zhang, Di Yin, Xing Sun, et al. Mac-sql: A multi-agent collaborative framework for text-to-sql. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 540–557, 2025.
- [111] Huichen Will Wang, Larry Birnbaum, and Vidya Setlur. Jupybara: Operationalizing a design space for actionable data analysis and storytelling with llms. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pages 1–24, 2025.
- [112] William Yang Wang. "liar, liar pants on fire": A new benchmark dataset for fake news detection. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Association for Computational Linguistics, 2017.
- [113] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [114] Liwenhan Xie, Chengbo Zheng, Haijun Xia, Huamin Qu, and Chen Zhu-Tian. Waitgpt: Monitoring and steering conversational llm agent in data analysis with on-the-fly code visualization. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, pages 1–14, 2024.
- [115] Zhuohan Xie, Rui Xing, Yuxia Wang, Jiahui Geng, Hasan Iqbal, Dhruv Sahnan, Iryna Gurevych, and Preslav Nakov. Fire: Fact-checking with iterative retrieval and verification. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 2901–2914, 2025.

- [116] Cheng Xiong, Gengfeng Zheng, Xiao Ma, Chunlin Li, and Jiangfeng Zeng. Delphiagent: A trustworthy multi-agent verification framework for automated fact verification. *Information Processing & Management*, 62(6):104241, 2025.
- [117] Shuo Yang, Yuqin Dai, Guoqing Wang, Xinran Zheng, Jinfeng Xu, Jinze Li, Zhenzhe Ying, Weiqiang Wang, and Edith CH Ngai. Realfactbench: A benchmark for evaluating large language models in real-world fact-checking. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 13435–13441, 2025.
- [118] Yingxuan Yang, Huacan Chai, Shuai Shao, Yuanyi Song, Siyuan Qi, Renting Rui, and Weinan Zhang. Agentnet: Decentralized evolutionary coordination for LLM-based multi-agent systems. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=tXqLxHlb8Z>.
- [119] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 2369–2380, 2018.
- [120] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.
- [121] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii, editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3911–3921. Association for Computational Linguistics, 2018.
- [122] Tao Yu, Rui Zhang, Heyang Er, Suyi Li, Eric Xue, Bo Pang, Xi Victoria Lin, Yi Chern Tan, Tianze Shi, Zihan Li, et al. Cosql: A conversational text-to-sql challenge towards cross-domain natural language interfaces to databases. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 1962–1979, 2019.
- [123] Jipeng Zhang, Haolin Yang, Kehao Miao, Ruiyuan Zhang, Renjie Pi, Jiahui Gao, and Xiaofang Zhou. ExeSQL: Self-taught text-to-SQL models with execution-driven bootstrapping for SQL dialects. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 24305–24326, 2025.
- [124] Shiyao Zhang, Yuji Dong, Yichuan Zhang, Terry R Payne, and Jie Zhang. Large language model assisted multi-agent dialogue for ontology alignment. In *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*, pages 2594–2596, 2024.
- [125] Tianshu Zhang, Kun Qian, Siddhartha Sahai, Yuan Tian, Shaddy Garg, Huan Sun, and Yunyao Li. Evoschema: Towards text-to-sql robustness against schema evolution. *Proceedings of the VLDB Endowment*, 18(10):3655–3668, 2025.
- [126] Wenqi Zhang, Yongliang Shen, Weiming Lu, and Yueting Zhuang. Data-copilot: Bridging billions of data and humans with autonomous workflow. In *ICLR 2024 Workshop on Large Language Model (LLM) Agents*, 2024.
- [127] Xinjie Zhao, Moritz Blum, Fan Gao, Yingjian Chen, Boming Yang, Luis Marquez-Carpintero, Mónica Pina-Navarro, Yanran Fu, So Morikawa, Yusuke Iwasawa, et al. Agentigraph: A multi-agent knowledge graph framework for interactive, domain-specific llm chatbots. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, pages 6757–6761, 2025.
- [128] Yuheng Zhao, Junjie Wang, Linbing Xiang, Xiaowen Zhang, Zifei Guo, Cagatay Turkay, Yu Zhang, and Siming Chen. Lightva: Lightweight visual analytics with llm agent-based task planning and execution. *IEEE Transactions on Visualization and Computer Graphics*, 31:6162–6177, 2024.
- [129] Victor Zhong, Caiming Xiong, and Richard Socher. Seq2sql: Generating structured queries from natural language using reinforcement learning. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1405–1415. Association for Computational Linguistics, 2017. URL <https://aclanthology.org/P17-1239>.
- [130] Victor Zhong, Caiming Xiong, and Richard Socher. Seq2SQL: Generating structured queries from natural language using reinforcement learning, 2018. URL <https://openreview.net/forum?id=Syx6bz-Ab>.
- [131] Dongzhuoran Zhou, Yuqicheng Zhu, Xiaxia Wang, Hongkuan Zhou, Jiaoyan Chen, Steffen Staab, Yuan He, and Evgeny Kharlamov. Gr-agent: Adaptive graph reasoning agent under incomplete knowledge. *arXiv preprint arXiv:2512.14766*, 2025.
- [132] Xuanhe Zhou, Junxuan He, Wei Zhou, Haodong Chen, Zirui Tang, Haoyu Zhao, Xin Tong, Guoliang Li, Youmin Chen, Jun Zhou, et al. A survey of llm and data. *arXiv preprint arXiv:2505.18458*, 2025.
- [133] Yizhang Zhu, Liangwei Wang, Chenyu Yang, Xiaotian Lin, Boyan Li, Wei Zhou, Xinyu Liu, Zhangyang Peng, Tianqi Luo, Yu Li, et al. A survey of data agents: Emerging paradigm or overstated hype? *arXiv preprint arXiv:2510.23587*, 2025.
- [134] Zipstack. Unstruct: Llm-driven extraction of unstructured data. <https://github.com/Zipstack/unstruct>, 2025. Open-source platform for extracting structured data from unstructured documents using LLMs.